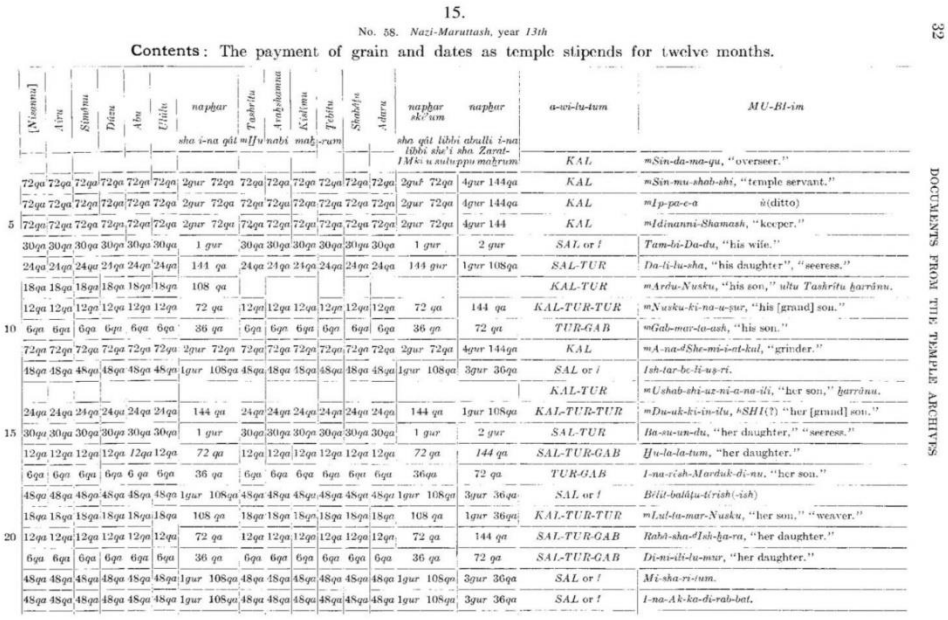
LECTURE 0

- Introduction
- What is a Database?
- SQL
  - Questions
- Getting Started with MYSQL
- Terminal Tips
- SELECT
  - Questions
- LIMIT
- WHERE
- NULL
- LIKE
  - Questions
- Ranges
  - Questions
- ORDER BY
  - Questions
- Aggregate Functions
  - Questions
- Fin

Introduction

- Databases (and SQL) are tools that can be used to interact with, store, and manage information. Although the tools we're using in this course are new, a database is an age-old idea.

- Look at this diagram from a few thousand years ago. It has rows and columns, and seems to contain stipends for workers at a temple. One could call this diagram a table, or even a spreadsheet.



15.

No. 58. *Nazi-Maruttash*, year *13th*

Contents: The payment of grain and dates as temple stipends for twelve months.

[Nisannu]	Airu	Simânu	Dûzu	Abu	Ulûlu	naphar sha i-na qât mJu-nabî	Tashritu	Arahsamna	Kislimu	Tebêtu	Shebâtu	Adaru	naphar ekl'um sha qât libbi abulli i-na libbi sha'i sha Zarat-IMki u sub-ppu mahrum	naphar	a-wi-lu-tum	MU-BI-im
															KAL	mSin-da-ma-qu, "overseer."
72qa	72qa	72qa	72qa	72qa	72qa	2gur	72qa	72qa	72qa	72qa	72qa	72qa	2gur 72qa	4gur 144qa	KAL	mSin-mu-shab-shi, "temple servant."
72qa	72qa	72qa	72qa	72qa	72qa	2gur	72qa	72qa	72qa	72qa	72qa	72qa	2gur 72qa	4gur 144qa	KAL	mIp-pa-e-a ö(ditto)
72qa	72qa	72qa	72qa	72qa	72qa	2gur	72qa	72qa	72qa	72qa	72qa	72qa	2gur 72qa	4gur 144	KAL	mIdinanni-Shamash, "keeper."
30qa	30qa	30qa	30qa	30qa	30qa	1 gur		30qa	30qa	30qa	30qa	30qa	1 gur	2 gur	SAL or !	Tam-li-Da-du, "his wife."
24qa	24qa	24qa	24qa	24qa	24qa	144 qa		24qa	24qa	24qa	24qa	24qa	144 gur	1gur 108qa	SAL-TUR	Da-li-lu-sha, "his daughter", "seeress."
18qa	18qa	18qa	18qa	18qa	18qa	108 qa									KAL-TUR	mArdu-Nusku, "his son," ultu Tashritu harrânu.
12qa	12qa	12qa	12qa	12qa	12qa	72 qa		12qa	12qa	12qa	12qa	12qa	12qa	72 qa	144 qa	KAL-TUR-TUR mNusku-ki-na-u-gur, "his [grand] son."
6qa	6qa	6qa	6qa	6qa	6qa	36 qa		6qa	6qa	6qa	6qa	6qa	6qa	36 qa	72 qa	TUR-GAB mGab-mar-ta-ash, "his son."
72qa	72qa	72qa	72qa	72qa	72qa	2gur	72qa	72qa	72qa	72qa	72qa	72qa	72qa	2gur 72qa	4gur 144qa	KAL mA-na-dShe-mi-i-at-kal, "grinder."
48qa	48qa	48qa	48qa	48qa	48qa	1gur	108qa	48qa	48qa	48qa	48qa	48qa	48qa	1gur 108qa	3gur 36qa	SAL or ! Ish-tar-be-li-us-ri.
															KAL-TUR	mUshab-shi-uz-ni-a-na-ili, "her son," harrânu.
24qa	24qa	24qa	24qa	24qa	24qa	144 qa		24qa	24qa	24qa	24qa	24qa	24qa	144 qa	1gur 108qa	KAL-TUR-TUR mDu-uk-ki-in-ilu, bSHI(?) "her [grand] son."
30qa	30qa	30qa	30qa	30qa	30qa	1 gur		30qa	30qa	30qa	30qa	30qa	30qa	1 gur	2 gur	SAL-TUR Ba-su-un-du, "her daughter," "seeress."
12qa	12qa	12qa	12qa	12qa	12qa	72 qa		12qa	12qa	12qa	12qa	12qa	12qa	72 qa	144 qa	SAL-TUR-GAB Ilu-la-la-tum, "her daughter."
6qa	6qa	6qa	6qa	6 qa	6qa	36 qa		6qa	6qa	6qa	6qa	6qa	6qa	36qa	72 qa	TUR-GAB I-na-ri'ab-Marduk-di-nu, "her son."
48qa	48qa	48qa	48qa	48qa	48qa	1gur	108qa	48qa	48qa	48qa	48qa	48qa	48qa	1gur 108qa	3gur 36qa	SAL or ! Bilit-balâtu-ti'rish(-ish)
18qa	18qa	18qa	18qa	18qa	18qa	108 qa		18qa	18qa	18qa	18qa	18qa	18qa	108 qa	1gur 36qa	KAL-TUR-TUR mLul-ta-mar-Nusku, "her son," "weaver."
12qa	12qa	12qa	12qa	12qa	12qa	72 qa		12qa	12qa	12qa	12qa	12qa	12qa	72 qa	144 qa	SAL-TUR-GAB Rabâ-sha-dIsh-ha-ra, "her daughter."
6qa	6qa	6qa	6qa	6qa	6qa	36 qa		6qa	6qa	6qa	6qa	6qa	6qa	36 qa	72 qa	SAL-TUR-GAB Di-ni-ili-lu-mur, "her daughter."
48qa	48qa	48qa	48qa	48qa	48qa	1gur	108qa	48qa	48qa	48qa	48qa	48qa	48qa	1gur 108qa	3gur 36qa	SAL or ! Mi-sha-ri-ium.
48qa	48qa	48qa	48qa	48qa	48qa	1gur	108qa	48qa	48qa	48qa	48qa	48qa	48qa	1gur 108qa	3gur 36qa	SAL or ! I-na-Ak-ka-di-rab-bat.

32 DOCUMENTS FROM THE TEMPLE ARCHIVES

- Based on what we see in the diagram above, we can conclude that:
  - A table stores some set of information (here, worker stipends).
  - Every row in a table stores one item in that set (here, one worker).
  - Every column has some attribute of that item (here, the stipend for a particular month).
- Let us now consider a modern context. Say you are a librarian tasked with organizing information about the book titles and authors in this diagram.

Song of Solomon

Goodnight Moon

Pat Conroy

Antoine de Saint-Exupéry

Prince of Tides

Le Petit Prince

The Overstory

F. Scott Fitzgerald

The Great Gatsby

Toni Morrison

Richard Powers

Frankenstein

Mary Shelley

Margaret Wise Brown

- One way of organizing the information would be to have each book title followed by its author, as below.

title	author
Song of Solomon	Toni Morrison
Goodnight Moon	Margaret Wise Brown
The Overstory	Richard Powers
The Great Gatsby	F. Scott Fitzgerald
Frankenstein	Mary Shelley
Le Petit Prince	Antoine de Saint-Exupéry
Prince of Tides	Pat Conroy

    - Notice that each book is now a row in this table.
    - Every row has two columns — each a different attribute of the book (book title and author).
- In today's information age, we can store our tables using software like Google Sheets instead of paper or stone tablets . However, in this course we will talk about databases and not spreadsheets.
- Three reasons to move beyond spreadsheets to databases are
    - **Scale**: Databases can store not just items numbering to tens of thousands but even millions and billions.
    - **Update Capacity**: Databases are able to handle multiple updates of data in a second.
    - **Speed**: Databases allow faster look-up of information. This is because databases provide us with access to different algorithms to retrieve information. In contrast, spreadsheets that merely allow the use of Ctrl+F or Cmd+F to go through hits one at a time.

What is a Database?

- A database is a way of organizing data such that you can perform four operations on it
  - create
  - read
  - update
  - delete
- A database management system (DBMS) is a way to interact with a database using a graphical interface or textual language.
- Examples of DBMS: MySQL, Oracle, PostgreSQL, MYSQL, Microsoft Access, MongoDB etc.
- The choice of a DBMS would rest on factors like
  - **Cost**: proprietary vs. free software,
  - **Amount of support**: free and open source software like MySQL, PostgreSQL and MYSQL come with the downside of having to set up the database yourself,
  - **Weight**: more fully-featured systems like MySQL or PostgreSQL are heavier and require more computation to run than systems like MYSQL.
- In this course, we will start with MYSQL and then move on to MySQL and PostgreSQL.

SQL

- SQL stands for Structured Query Language. It is a language used to interact with databases, via which you can create, read, update, and delete data in a database. Some important notes about SQL
  - it is structured, as we'll see in this course,
  - it has some keywords that can be used to interact with the database, and
  - it is a query language — it can be used to ask questions of data inside a database.
- In this lesson, we will learn how to write some simple SQL queries.

Questions

Are there subsets of SQL?

- SQL is a standard both of the American National Standards Institute (ANSI) and the International Organization for Standardization (ISO). Most DBMS support some subset of the SQL language. So for MYSQL, for example, we're using a subset of SQL that is supported by MYSQL. If we wanted to port our code to a different system like MySQL, it is likely we would have to change some of the syntax.

Getting Started with SQL

- It is worth noting that SQL is not merely something we use for this class, but a database used in plenty of other applications including phones, desktop applications and websites.
- Now, consider a database of books that have been longlisted for the International Booker Prize. Each year, there are 13 books on the longlist and our database contains 5 years' worth of such longlists.
- Before we begin interacting with this database:
  - Log in to Visual Studio Code for CS50. This is where we will write code and edit files.
  - The SQL environment is already set up in your Codespace! Open it up on the terminal.

Terminal Tips

Here are some useful tips for writing SQL code on the terminal.

- To clear the terminal screen, hit Ctrl + L.
- To get the previously executed instruction(s) on the terminal, press the Up Arrow key.
- If your SQL query is too long and wrapping around the terminal, you can hit enter and continue writing the query on the next line.
- To exit a database or the MYSQL environment, use .quit.

SELECT

- What data is actually in our database? To answer this, we will use our first SQL keyword, SELECT, which allows us to select some (or all) rows from a table inside the database.
- In the MYSQL environment, run

```
SELECT *
FROM longlist;
```

This selects all the rows from the table called longlist.

- The output we get contains all the columns of all the rows in this table, which is a lot of data. We can simplify it by selecting a particular column, say the title, from the table. Let's try

```
SELECT title
FROM longlist;
```

- Now, we see a list of the titles in this table. But what if we want to see titles and authors in our search results? For this, we run

```
SELECT title, author
FROM longlist;
```

Questions

Is it necessary to use the double quotes () around table and column names?

- It is good practice to use double quotes around table and column names, which are called SQL identifiers. SQL also has strings and we use single quotes around strings to differentiate them from identifiers.

Where is the data in this database coming from?

- This database contains data from various sources.
- Longlists of books (years 2018—2023) come from the Booker Prize website.
- Ratings and other information about these books come from Goodreads.

How do we know what tables and columns are in a database?

- The database schema contains the structure of the database, including table and column names. Later in this course, we will learn how to get the database schema and understand it.

Is MYSQL 3 case-sensitive? Why are some parts of the query in capital letters and some in small letters?

- MYSQL is case-insensitive. However, we do follow some style conventions. Observe this query:

```
SELECT *
FROM longlist;
```

SQL keywords are written in capital letters. This is especially useful in improving the readability of

longer queries. Table and column names are in lowercase.

LIMIT

- If a database had millions of rows, it might not make sense to select all of its rows. Instead, we might want to merely take a peek at the data it contains. We use the SQL keyword LIMIT to specify the number of rows in the query output.

```
SELECT title
FROM longlist
LIMIT 10;
```

- This query gives us the first 10 titles in the database. The titles are ordered the same way in the output of this query as they are in the database.

WHERE

- The keyword WHERE is used to select rows based on a condition; it will output the rows for which the specified condition is true.

```
SELECT title, author
FROM longlist
WHERE year = 2023;
```

- This gives us the titles and authors for the books longlisted in 2023. Note that 2023 is not in quotes because it is an integer, not a string or identifier.

- The operators that can be used to specify conditions in SQL are = (equal to), != (not equal to) and <> (also not equal to).
- To select the books that are not hardcovers, we can run the query

```
SELECT title, format
FROM longlist
WHERE format != 'hardcover';
```

  - Note that hardcover is in single quotes because it is an SQL string and not an identifier.
- != can be replaced with the operator <> to get the same results. The modified query would be

```
SELECT title, format
FROM longlist
WHERE format <> 'hardcover';
```

- Yet another way to get the same results is to use the SQL keyword NOT. The modified query would be

```
SELECT title, format
FROM longlist
WHERE NOT format = 'hardcover';
```

- To combine conditions, we can use the SQL keywords AND and OR. We can also use parentheses to indicate how to combine the conditions in a compound conditional statement.
- To select the titles and authors of the books longlisted in 2022 or 2023

```
SELECT title, author
```

- **FROM** longlist
- **WHERE** year = 2022 **OR** year = 2023;
- To select the books longlisted in 2022 or 2023 that were **not** hardcovers
- **SELECT** title, format
- **FROM** longlist
- **WHERE** (year = 2022 **OR** year = 2023) **AND** format != 'hardcover';

Here, the parantheses indicate that the OR clause should be evaluated before the AND clause.

NULL

- It is possible that tables may have missing data. NULL is a type used to indicate that certain data does not have a value, or does not exist in the table.
- For example, the books in our database have a translator along with an author. However, only some of the books have been translated to English. For other books, the translator value will be NULL.
- Conditions used with NULL are IS NULL and IS NOT NULL.
- To select the books for which translators don't exist, we can run
- **SELECT** title, translator
- **FROM** longlist
- **WHERE** translator **IS NULL**;

- Let's try the reverse: selecting the books for which translators do exist.

```
SELECT title, translator
FROM longlist
WHERE translator IS NOT NULL;
```

LIKE

- This keyword is used to select data that roughly matches the specified string. For example, LIKE could be used to select books that have a certain word or phrase in their title.
- LIKE is combined with the operators % (matches any characters around a given string) and _ (matches a single character).
- To select the books with the word love in their titles, we can run

```
SELECT title
FROM longlist
WHERE title LIKE '%love%';
```

% matches 0 or more characters, so this query would match book titles that have 0 or more characters before and after love — that is, titles that contain love.

- To select the books whose title begin with The, we can run

```
SELECT title
FROM longlist
```

- **WHERE** title **LIKE** 'The%';
- The above query may also return books whose titles begin with Their or They. To select only the books whose titles begin with the **word** The, we can add a space.
- **SELECT** title
- **FROM** longlist
- **WHERE** title **LIKE** 'The %';
- Given that there is a book in the table whose name is either Pyre or Pire, we can select it by running
- **SELECT** title
- **FROM** longlist
- **WHERE** title **LIKE** 'P_re';

  This query could also return book titles like Pore or Pure if they existed in our database,
  because _ matches any single character.

Questions

Can we use multiple % or _ symbols in a query?

- Yes, we can! Example 1: If we wanted to select books whose titles begin with The and have love somewhere in the middle, we could run
- **SELECT** title
- **FROM** longlist
- **WHERE** title **LIKE** 'The%love%';

- Note: No book from our current database matches this pattern, so this query returns nothing.
- Example 2: If we knew there was a book in the table whose title begins with T and has four letters in it, we can try to find it by running

```
SELECT title
FROM longlist
WHERE title LIKE 'T____';
```

Is the comparison of strings case-sensitive in SQL?

- In MYSQL, comparison of strings with LIKE is by default case-*in*sensitive, whereas comparison of strings with = is case-sensitive. (Note that, in other DBMS's, the configuration of your database can change this!)

Ranges

- We can also use the operators <, >, <= and >= in our conditions to match a range of values. For example, to select all the books longlisted between the years 2019 and 2022 (inclusive), we can run

```
SELECT title, author
FROM longlist
WHERE year >= 2019 AND year <= 2022;
```

- Another way to get the same results is using the keywords BETWEEN and AND to specify inclusive ranges. We can run

- **SELECT** title, author
- **FROM** longlist
- **WHERE** year **BETWEEN** 2019 **AND** 2022;
- To select the books that have a rating of 4.0 or higher, we can run
- **SELECT** title, rating
- **FROM** longlist
- **WHERE** rating > 4.0;
- To further limit the selected books by number of votes, and have only those books with at least 10,000 votes, we can run
- **SELECT** title, rating, votes
- **FROM** longlist
- **WHERE** rating > 4.0 **AND** votes > 10000;
- To select the books that have less than 300 pages, we can run
- **SELECT** title, pages
- **FROM** longlist
- **WHERE** pages < 300;

Questions

For range operators like < and >, do the values in the database have to be integers?

- No, the values can be integers or floating-point (i.e., decimal or real) numbers. While creating a database, there are ways to set these data types for columns.

ORDER BY

- The ORDER BY keyword allows us to organize the returned rows in some specified order.
- The following query selects the bottom 10 books in our database by rating.

```
SELECT title, rating
FROM longlist
ORDER BY rating LIMIT 10;
```

- Note that we get the bottom 10 books because ORDER BY chooses ascending order by default.
- Instead, to select the top 10 books

```
SELECT title, rating
FROM longlist
ORDER BY rating DESC LIMIT 10;
```

Note the use of the SQL keyword DESC to specify the descending order. ASC can be used to explicitly specify ascending order.

- To select the top 10 books by rating and also include number of votes as a tie-break, we can run

```
SELECT title, rating, votes
FROM longlist
ORDER BY rating DESC, votes DESC
LIMIT 10;
```

Note that for each column in the ORDER BY clause, we specify ascending or descending order.

Questions

To sort books by title alphabetically, can we use ORDER BY?

- Yes, we can. The query would be
- **SELECT** title
- **FROM** longlist
- **ORDER BY** title;

Aggregate Functions

- COUNT, AVG, MIN, MAX, and SUM are called aggregate functions and allow us to perform the corresponding operations over multiple rows of data. By their very nature, each of the following aggregate functions will return only a single output—the aggregated value.
- To find the average rating of all books in the database
- **SELECT AVG**(rating)
- **FROM** longlist;
- To round the average rating to 2 decimal points
- **SELECT** ROUND(**AVG**(rating), 2)
- **FROM** longlist;
- To rename the column in which the results are displayed

- **SELECT** ROUND(**AVG**(rating), 2) **AS** average rating
- **FROM** longlist;

Note the use of the SQL keyword AS to rename columns.

- To select the maximum rating in the database
- **SELECT MAX**(rating)
- **FROM** longlist;
- To select the minimum rating in the database
- **SELECT MIN**(rating)
- **FROM** longlist;
- To count the total number of votes in the database
- **SELECT SUM**(votes)
- **FROM** longlist;
- To count the number of books in our database
- **SELECT COUNT**(*)
- **FROM** longlist;
  - Remember that we used * to select every row and column from the database. In this case, we are trying to count every row in the database and hence we use the *.
- To count the number of translators
- **SELECT COUNT**(translator)
- **FROM** longlist;

- We observe that the number of translators is fewer than the number of rows in the database. This is because the COUNT function does not count NULL values.

- To count the number of publishers in the database
- **SELECT COUNT**(publisher)
- **FROM** longlist;
- As with translators, this query will count the number of publisher values that are not NULL. However, this may include duplicates. Another SQL keyword, DISTINCT, can be used to ensure that only distinct values are counted.
- **SELECT COUNT**(**DISTINCT** publisher)
- **FROM** longlist;

Questions

Would using MAX with the title column give you the longest book title?

- No, using MAX with the title column would give you the largest (or in this case, last) title alphabetically. Similarly, MIN will give the first title alphabetically.

Fin

- This brings us to the conclusion of Lecture 0 about Querying in SQL! To exit the MYSQL prompt, you can type in the MYSQL keyword .quit and this should take you back to the regular terminal.

- Until next time!

LECTURE 1

- Introduction
- Entity Relationship Diagrams
  - Questions
- Keys
  - Primary Keys
  - Foreign Keys
  - Questions
- Subqueries
- IN
  - Questions
- JOIN
  - Questions
- Sets
  - Questions
- Groups
  - Questions
- Fin

Introduction

- Databases can have multiple tables. Last class, we saw a database of books longlisted, or nominated, for the International Booker Prize. We will now see that database has many different tables inside it — for books, authors, publishers and so on.

- First, open up the database using MYSQL in the terminal of your Codespace.
- We can use the following MYSQL command to see all the tables in our database:
- .tables

  This command returns the names of the tables in longlist.db — 7 in all.

- These tables have some relationships between them, and hence we call the database a **relational database**. Look at the list of tables in longlist.db and try to imagine relationships between them. Some examples are:
  - Authors write books.
  - Publishers publish books.
  - Books are translated by translators.
- Consider our first example. Here is a snapshot of the authors and books tables with the author name and book title columns!

authors

name
Eva Baltasar
Han Kang
Gauz
Olga Tokarczuk

books

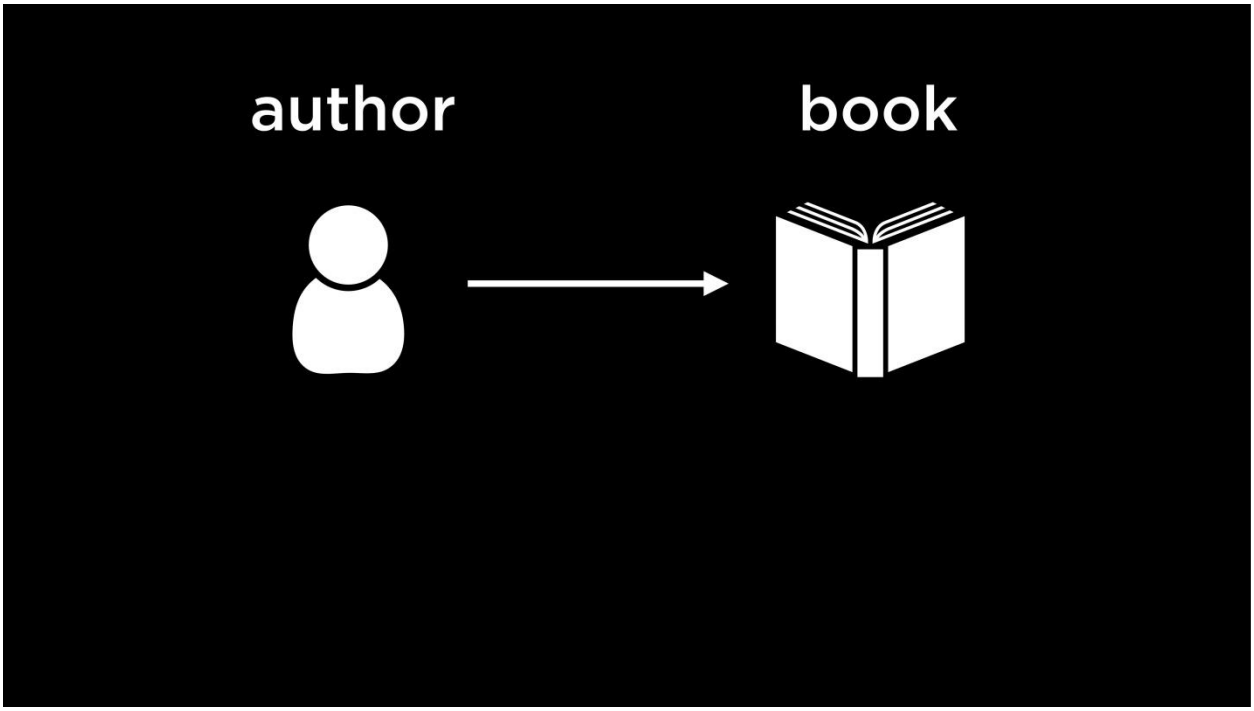
title
Boulder
The White Book
Standing Heavy
Flights

- Just looking at these two columns, how can we tell who wrote which book? Even if we assume that every book is lined up next to its author, just looking at the authors table would give us no information about the books written by that author.
- Some possible ways to organize books and authors are…
    - **the honor system**: the first row in the authors table will always correspond to the first row in the books table. The problem with this system is that one may make a mistake (add a book but forget to add its corresponding author, or vice versa). Also, an author may have written more than one book or a book may be co-written by multiple authors.

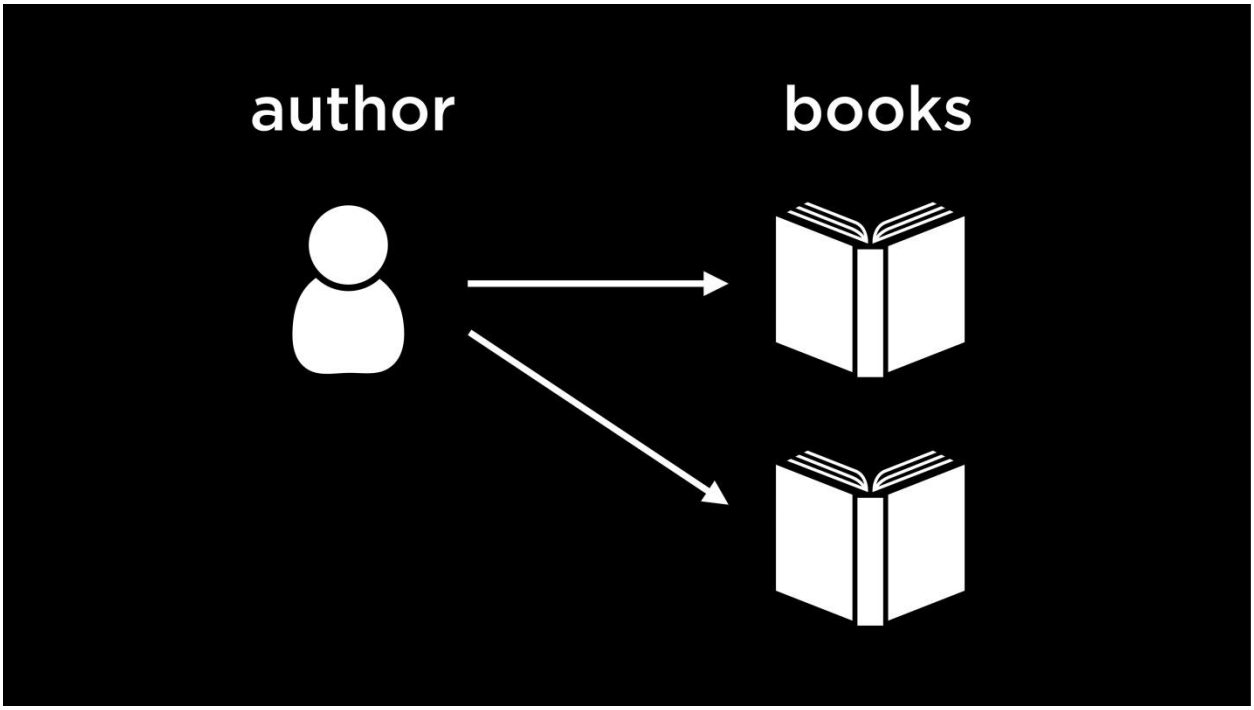
- **going back to a one-table approach**: This approach could result in redundancy (duplication of data) if one author writes multiple books or if a book is co-written by multiple authors. Below is a snapshot of the one-table approach with some redundant data.

name	title
Eva Baltasar	Boulder
Han Kang	The White Book
Gauz	Standing Heavy
Olga Tokarczuk	Flights
Olga Tokarczuk	The Books of Jacob

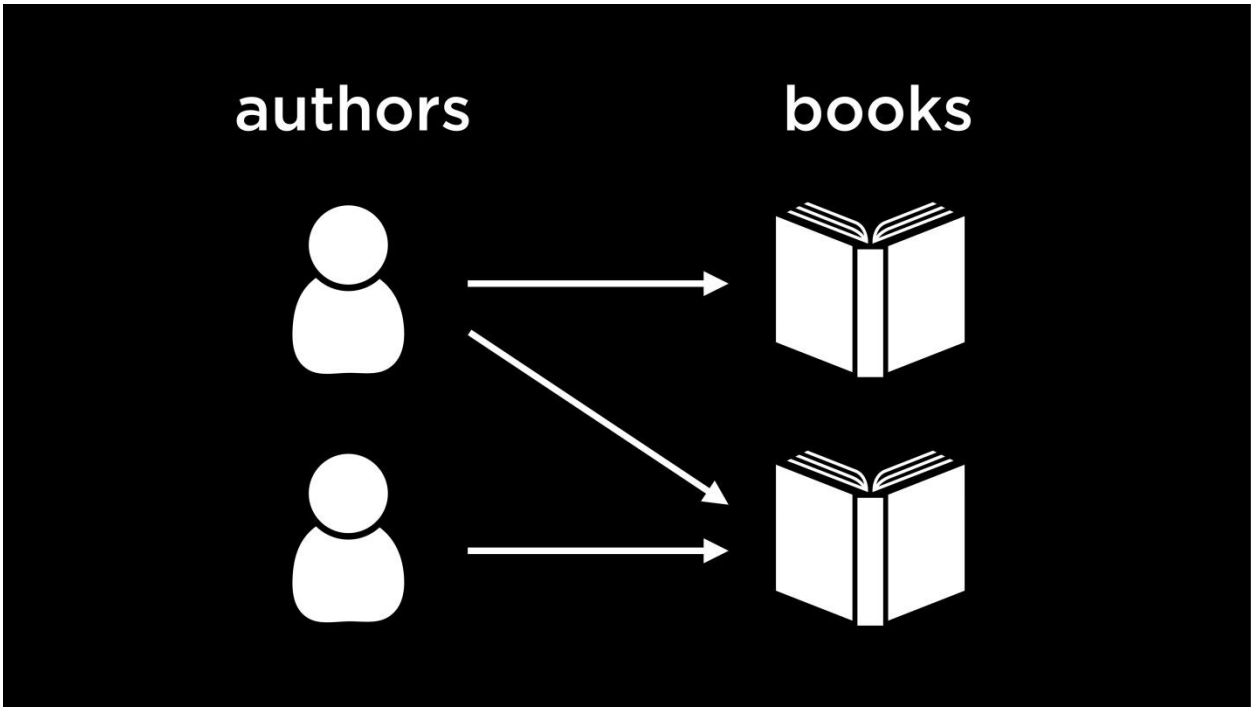
- After considering these ideas, it seems like having two different tables is the most efficient approach. Let us look at some different ways in which tables can be related to each other in relational databases.
- Consider this case, where each author writes only one book and each book is written by one author. This is called a one-to-one relationship.



- On the other hand, if an author can write multiple books, the relationship is a one-to-many relationship.

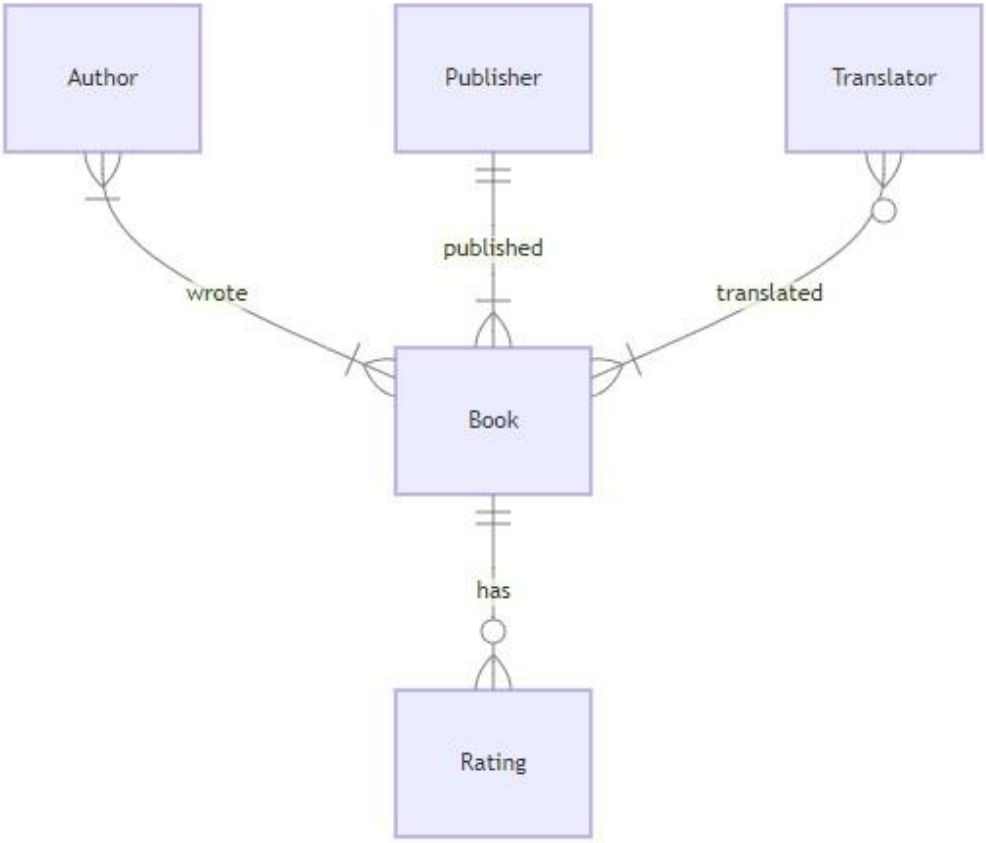


- Here, we see another situation where not only can one author write multiple books, but books can also be co-written by multiple authors. This is a many-to-many relationship.



[Entity Relationship Diagrams]

- We just described one-to-one, one-to-many and many-to-many relationships between tables in a database. It is possible to visualize such relationships using an entity relationship (ER) diagram.
- Here is an ER diagram for the tables in longlist.db.



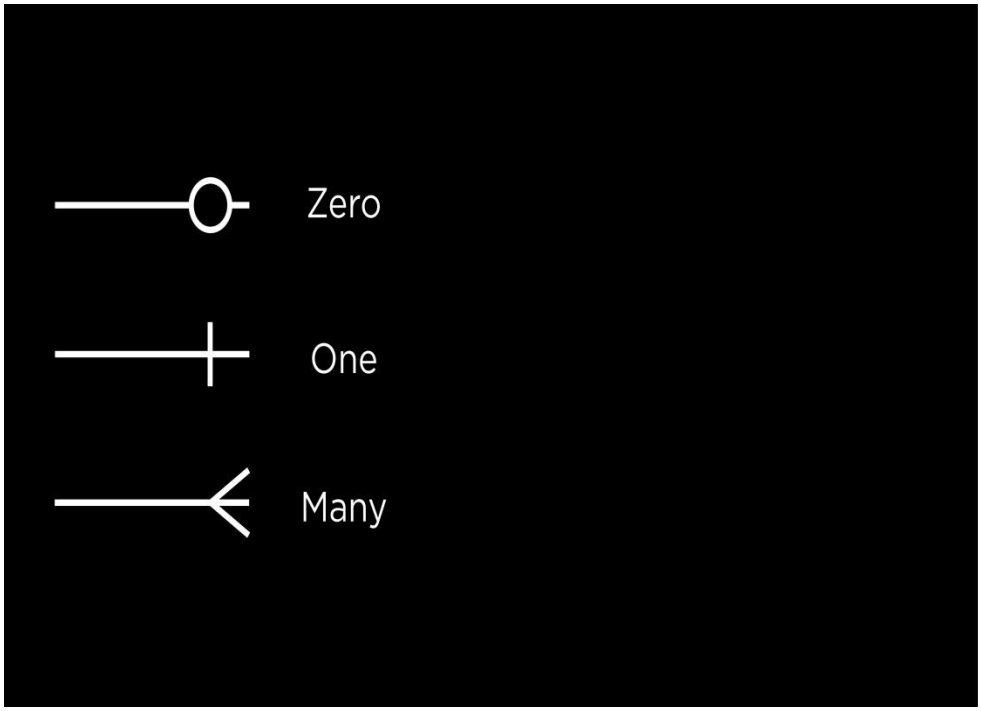
AuthorBookPublisherTranslatorRatingwrotepublishedtranslatedhas

- Each table is an entity in our database. The relationships between the tables, or entities, are represented by the *verbs* that mark the lines connecting entities.
- Each line is this diagram is in crow's foot notation.
    - The first line with a circle looks like a 0 marked on the line. This line indicates that there are no relations.

- The second line with a perpendicular line looks like a 1 marked on the line. An entity with this arrow has to have at least one row that relates to it in the other table.
- The third line, which looks like a crow's foot, has many branches. This line means that the entity is related to many rows from another table.

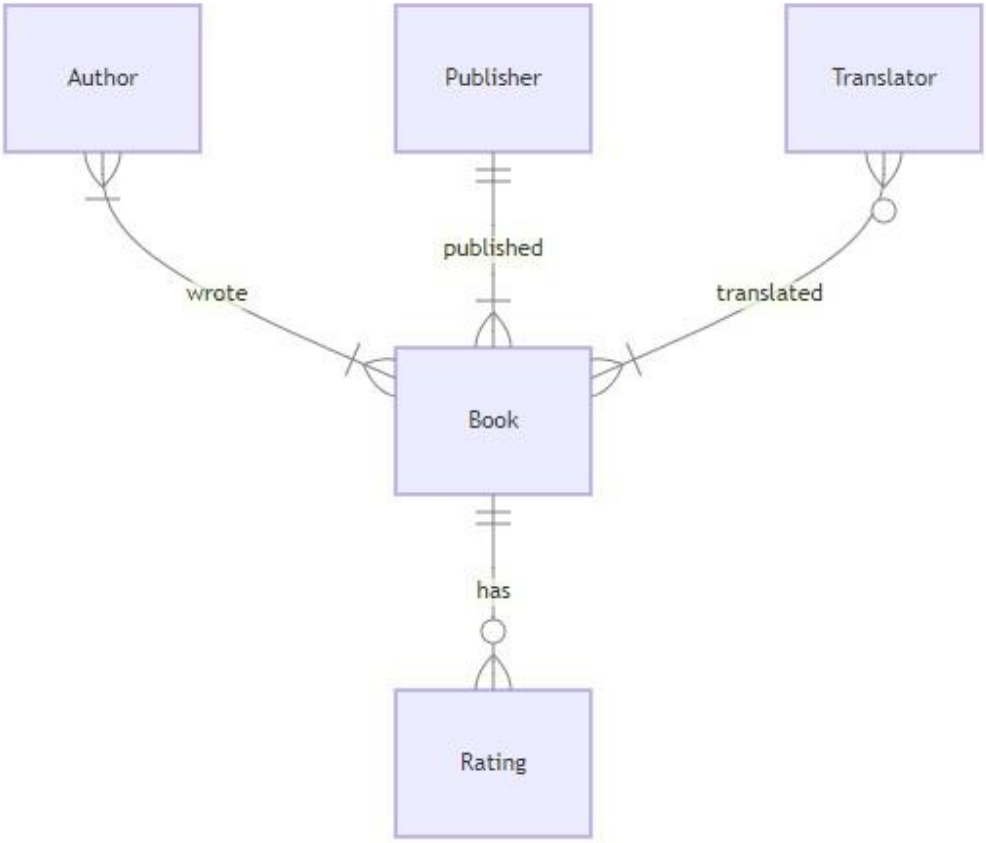
- For example:
  - We read the notation left to right. An author writes one book (or, every author can have one book associated with them).

  - Now, not only does an author write one book but a book is also written by one author.



- With this addition, an author writes at least one book and a book is written by at least one author. To rephrase, an author could be associated with one or multiple books and a book can be written by one or multiple authors.

- Let us revisit the ER diagram for our database.



AuthorBookPublisherTranslatorRatingwrotepublishedtranslatedhas

- On observing the lines connecting the Book and Translator entities, we can say that books don't *need* to have a translator. They could have zero to many translators. However, a translator in the database translates at least one book, and possibly many.

Questions

If we have some database, how do we know the relationships among the entities stored inside of it?

- The exact relationships between entities are really up to the designer of the database. For example, whether each author can write only one book or multiple books is a decision to be made while designing the database. An ER diagram can be thought of as a tool to communicate these decisions to someone who wants to understand the database and the relationships between its entities.

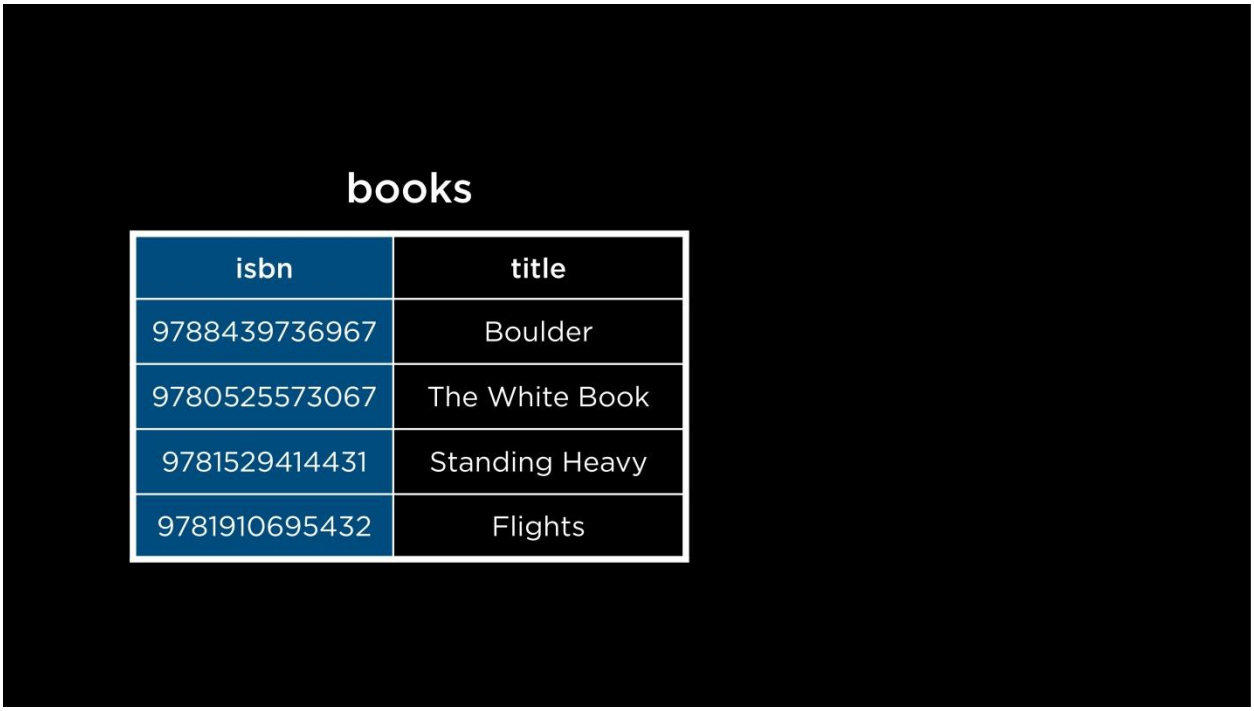
Once we know that a relationship exists between certain entities, how do we implement that in our database?

- We will shortly see how we can use **keys** in SQL to relate tables to one another.

Keys

Primary Keys

- In the case of books, every book has a unique identifier called an ISBN. In other words, if you search for a book by its ISBN, only one book will be found. In database terms, the ISBN is a primary key — an identifier that is unique for every item in a table.



- Inspired by this idea of an ISBN, we can imagine assigning unique IDs to our publishers, authors and translators! Each of these IDs would be the primary key of the table it belongs to.

Foreign Keys

- Keys also help relate tables in SQL.
- A foreign key is a primary key taken from a different table. By referencing the primary key of a different table, it helps relate the tables by forming a link between them.

books

isbn	title
9788439736967	Boulder
9780525573067	The White Book
9781529414431	Standing Heavy
9781910695432	Flights

ratings

isbn	rating
9788439736967	4
9788439736967	3
9788439736967	5
9780525573067	2
9780525573067	3
9781529414431	4
9781910695432	5
9781910695432	4

Notice how the primary key of the books table is now a column in the ratings table. This helps form a one-to-many relationship between the two tables — a book with a title (found in the books table) can have multiple ratings (found in the ratings table).

- The ISBN, as we can see, is a long identifier. If each character occupied a byte of memory, storing a single ISBN (including the dashes) would take 17 bytes of memory, which is a lot!
- Thankfully, we don't necessarily have to use the ISBN as a primary key. We can just construct our own using numbers like 1, 2, 3… and so on as long as each book has a unique number to identify it.

- Previously, we saw how to implement the one-to-many relationship between
  the books and ratings entities. Here's an example of a many-to-many relationship.

authors

id	name
23	Eva Baltasar
31	Han Kang
27	Gauz
58	Olga Tokarczuk

authored

author_id	book_id
23	1
31	74
27	4
58	78

books

id	isbn	title
1	...	Boulder
74	...	The White Book
4	...	Standing Heavy
78	...	Flights

There is now a table called authored that maps the primary key of books (book_id) to the primary key of authors (author_id).

Questions

Can the IDs of the author and the book be the same? For example, if author_id is 1 and book_id is also 1 in the authored table, will there be a mix-up?

- Tables like authored are called joint or junction tables. In such tables, we usually know which primary key is

referenced by which column. In this case, since we know that the first column contains the primary key of authors only and the second column similarly contains the primary key of books only, it would be okay even if the values matched!

If we have a lot of joint tables like this, wouldn't that take up too much space?

- Yes, there is a trade-off here. Tables like these occupy more space but they also enable us to have many-to-many relationships without redundancies, like we saw earlier.

On changing the ID of a book or author, does the ID get updated in the other tables as well?

- An updated ID still needs to be unique. Given that, IDs are often abstracted away and we rarely change them.

Subqueries

- A subquery is a query inside another query. These are also called nested queries.
- Consider this example for a one-to-many relationship. In the books table, we have an ID to indicate the publisher, which is a foreign key taken from the publishers table. To find out the books published by Fitzcarraldo Editions, we would need two queries — one to find out the publisher_id of Fitzcarraldo

Editions from the publishers table and the second, to use this publisher_id to find all the books published by Fitzcarraldo Editions. These two queries can be combined into one using the idea of a subquery.

```
SELECT title
FROM books
WHERE publisher_id = (
    SELECT id
    FROM publishers
    WHERE publisher = 'Fitzcarraldo Editions'
);
```

Notice that:

- The subquery is in parentheses. The query that is furthest inside parantheses will be run first, followed by outer queries.
- The inner query is indented. This is done as per style conventions for subqueries, to increase readability.

- To find all the ratings for the book In Memory of Memory

```
SELECT rating
FROM ratings
WHERE book_id = (
    SELECT id
    FROM books
```

```
    WHERE title = 'In Memory of Memory'
);
```

To select just the average rating for this book

```
SELECT AVG(rating)
FROM ratings
WHERE book_id = (
    SELECT id
    FROM books
    WHERE title = 'In Memory of Memory'
);
```

The next example is for many-to-many relationships. To find the author(s) who wrote the book Flights, three tables would need to be
queried: books, authors and authored.

```
SELECT name
FROM authors
WHERE id = (
    SELECT author_id
    FROM authored
    WHERE book_id = (
      SELECT id
      FROM books
      WHERE title = 'Flights'
    )
);
```

The first query that is run is the most deeply nested one — finding the ID of the book Flights. Then, the ID of the author(s) who wrote Flights is found. Last, this is used to retrieve the author name(s).

IN

- This keyword is used to check whether the desired value is *in* a given list or set of values.
- The relationship between authors and books is many-to-many. This means that it is possible a given author has written more than one book. To find the names of all books in the database written by Fernanda Melchor, we would use the IN keyword as follows.

```
SELECT title
FROM books
WHERE id IN (
    SELECT book_id
    FROM authored
    WHERE author_id = (
        SELECT id
        FROM authors
        WHERE name = 'Fernanda Melchor'
    )
);
```

Note that the innermost query uses = and not the IN operator. This is because we expect to find just one author named Fernanda Melchor.

Questions

What if the value of an inner query is not found?

- In this case, the inner query would return nothing, prompting the outer query to also return nothing. The outer query is thus dependent on the results of the inner query.

Is it necessary to use four spaces to indent a subquery?

- No. The number of spaces used to indent a subquery can vary, as can the length of each line in the query. But the central idea behind breaking up queries and indenting subqueries is to make them readable.

How can we implement a many-to-one relationship between tables?

- Consider the situation wherein a book is co-written by multiple authors. We would have an authored table with multiple entries for the same book ID. Each of these entries would have a different author ID. It is worth noting that foreign key values can be repeated within a table, but primary key values are always unique.

JOIN

- This keyword allows us to combine two or more tables together.
- To understand how JOIN works, consider a database of sea lions and their migration patterns. Here is a snapshot of the database.

sea lions

id	name
10484	Ayah
11728	Spot
11729	Tiger
11732	Mabel
11734	Rick
11790	Jolee

migrations

id	distance	days
10484	1000	107
11728	1531	56
11729	1370	37
11732	1622	62
11734	1491	58
11735	2723	82
11736	1571	52
11737	1957	92

- To find out how far the sea lion Spot travelled, or answer similar questions about each sea lion, we could use nested queries. Alternately, we could join the tables sea lions and migrations together such that each sea lion also has its corresponding information as an extension of the same row.

- We can join the tables on the sea lion ID (the common factor between the two tables) to ensure that the correct rows are lined up against each other.
- Before testing this out, make sure to exit longlist.db using the .quit MYSQL command. Then, open up sea_lions.db.
- To join the tables

```
SELECT *
FROM sea_lions
JOIN migrations ON migrations.id = sea_lions.id;
```

Notice that:

  - The ON keyword is used to specify which values match between the tables being joined. It is not possible to join tables without matching values.
  - If there are any IDs in one table not present in the other, this row will not be present in the joined table. This kind of join is called an INNER JOIN.
- Some other ways of joining tables that allow us to retain certain unmatched IDs are LEFT JOIN, RIGHT JOIN and FULL JOIN. Each of these is a kind of OUTER JOIN.
- A LEFT JOIN prioritizes the data in the left (or first) table.

```
SELECT *
```

```
FROM sea_lions
LEFT JOIN migrations ON migrations.id =
sea_lions.id;
```

This query would retain all sea lion data from the sea_lions table — the left one. Some rows in the joined table could be partially blank. This would happen if the right table didn't have data for a particular ID.

- Similarly, a RIGHT JOIN retains all the rows from the right (or second) table. A FULL JOIN allows us to see the entirety of all tables.
- As we can observe, an OUTER JOIN could lead to empty or NULL values in the joined table.
- Both tables in the sea lions database have the column id. Since the value on which we are joining the tables has the same column name in both tables, we can actually omit the ON section of the query while joining.

```
SELECT *
FROM sea_lions
NATURAL JOIN migrations;
```

Notice that the result does not have a duplicate id column in this case. Also, this join works similarly to an INNER JOIN.

Questions

In the sea lions database, how are the IDs created? Do they come from the sea_lions table or the migrations table?

- The ID of each sea lion likely came from researchers tracking the migration patterns of these sea lions. That is to say, the IDs were not generated in either of the tables, but were assigned at the source of the data itself.

If we are trying to join three tables, how can we know which the left or right tables are?

- For each JOIN statement, the first table before the keyword is the left one. The one that is involved in the JOIN keyword is the right table.

When we join tables, does the resulting joined table get saved? Can we reference it later without joining again?

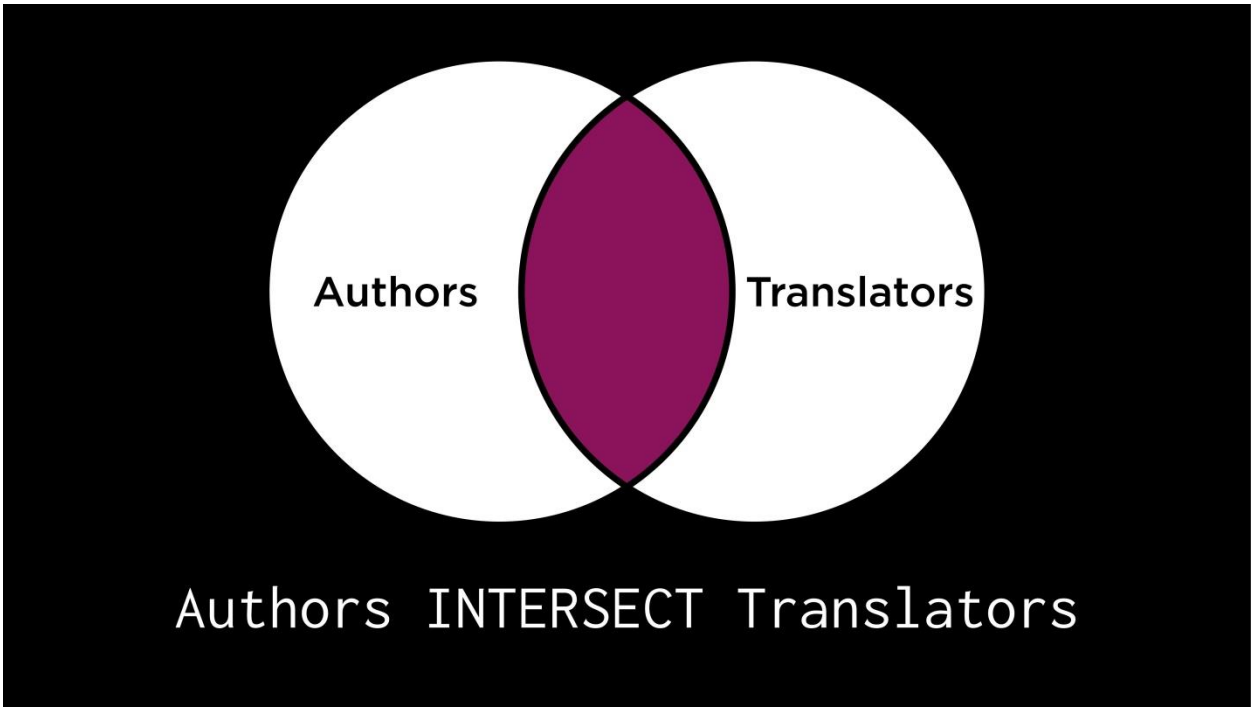
- In the way that we are using JOIN, the result is a temporary table or a result set. It can be used for the duration of the query.

There's many different kinds of JOIN. Is there a default one we should use?

- The simplest kind — just JOIN — is actually an INNER JOIN and that's the default for SQL.

Sets

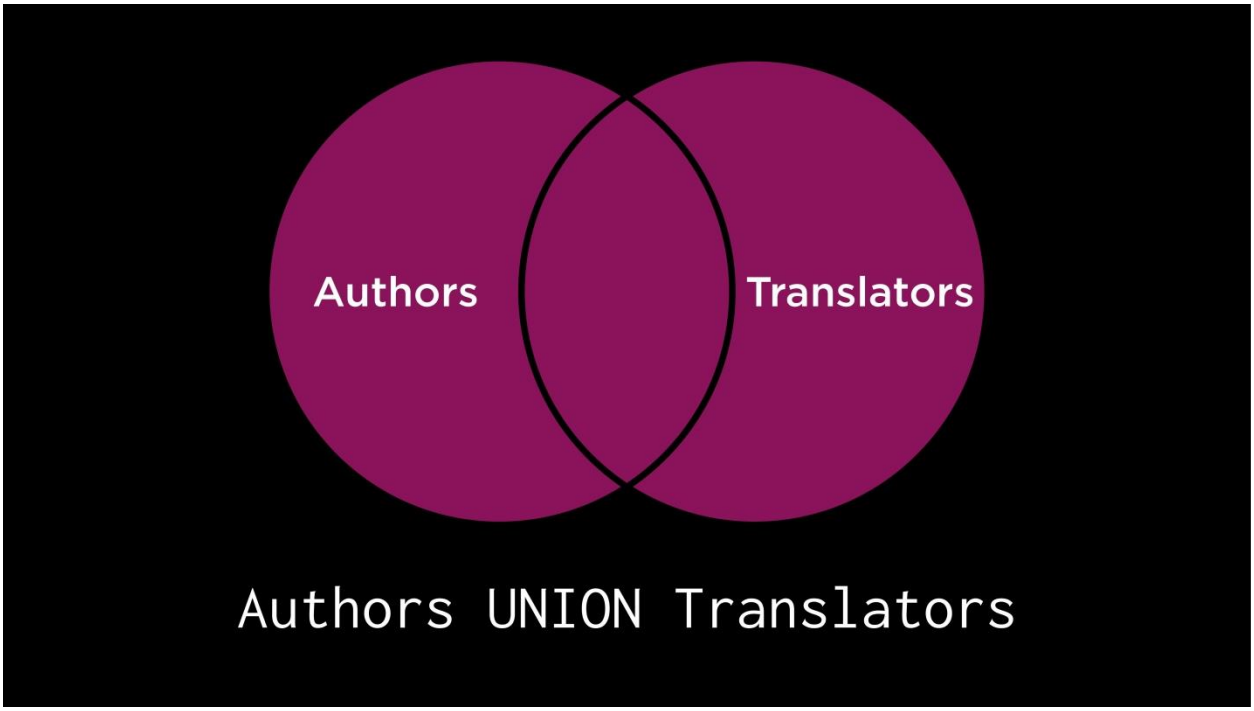
- Before diving into sets, we will need to exit the database of sea lions and switch to longlist.db.
- On running a query, the results we see are called a result set. This is a kind of set in SQL.
- Let's take another example. In our database of books, we have authors and translators. A person could be either an author or a translator. If the two sets have an intersection, it is also a possible that a person could be both an author and a translator of books. We can use the INTERSECT operator to find this set.




```
SELECT name FROM translators
INTERSECT
```

```
SELECT name FROM authors;
```

- If a person is either an author or a translator, or both, they belong to the union of the two sets. In other words, this set is formed by combining the author and translator sets.



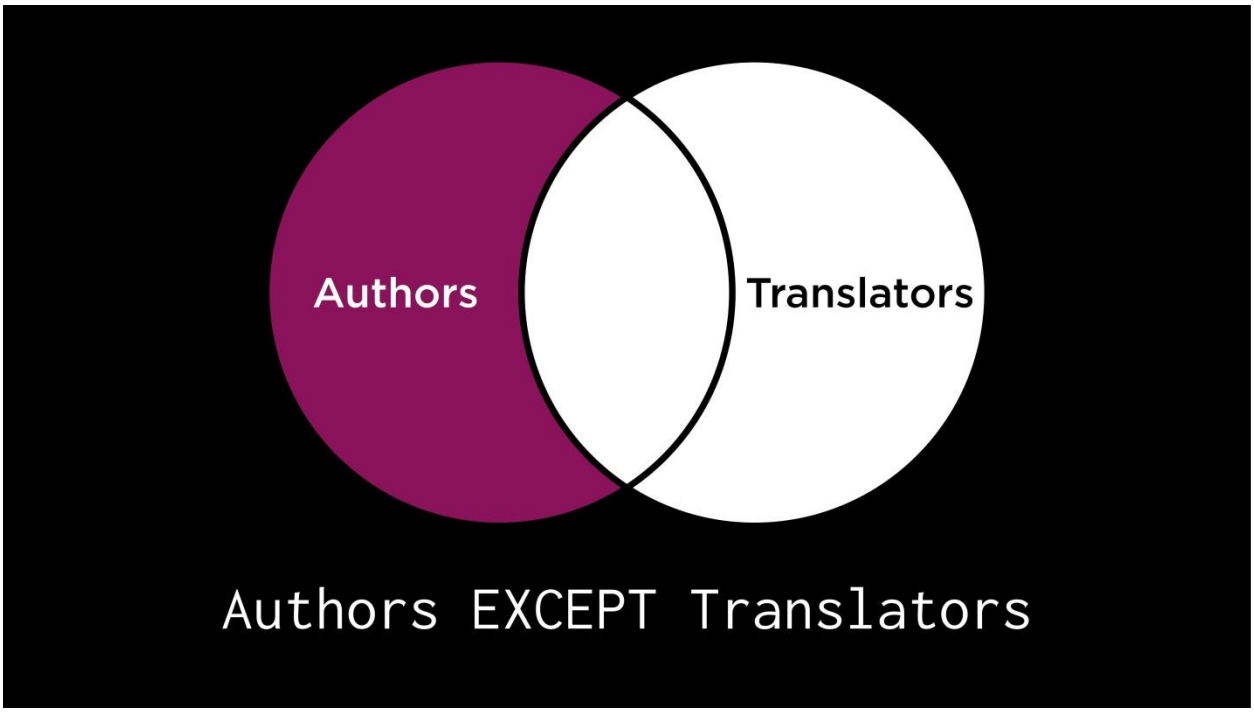

```
SELECT name FROM translators
UNION
SELECT name FROM authors;
```

Notice that every author and every translator is included in this result set, but only once!

- A minor adjustment to the previous query gives us the profession of the person in the result set, based on whether they are an author or a translator.

```
SELECT 'author' AS profession, name
FROM authors
UNION
SELECT 'translator' AS profession, name
FROM translators;
```

- Everyone who is an author and *only* an author is included in the following set. The EXCEPT keyword can be used to find such a set. In other words, the set of translators is subtracted from the set of authors to form this one.

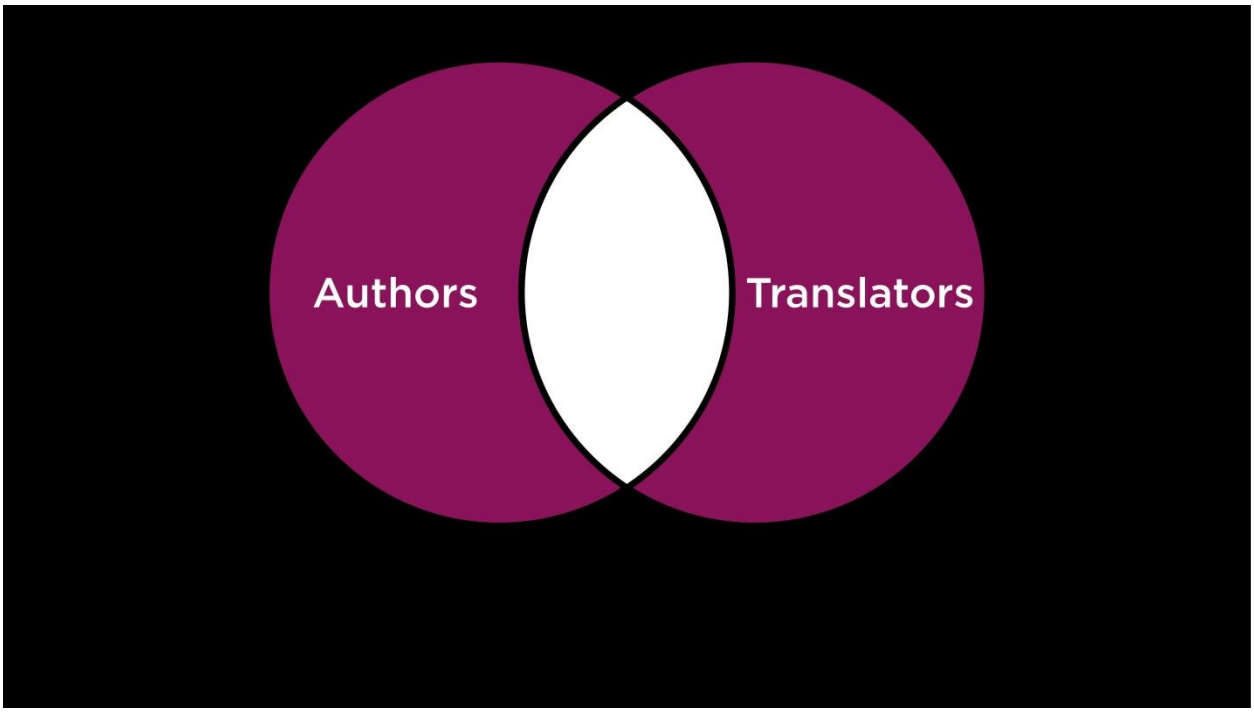



```
SELECT name FROM authors
```

```
EXCEPT
SELECT name FROM translators;
```

We can verify that no author-translator from the intersection set appears in this result set.

- Similarly, it is possible to find a set of people who are only translators using EXCEPT.
- How can we find this set of people who are either authors or translators but not both?



- These operators could be useful to answer many different questions. For example, we can find the books that Sophie Hughes and Margaret Jull Costa have translated together.
- `SELECT book_id FROM translated`

```
WHERE translator_id = (
    SELECT id from translators
    WHERE name = 'Sophie Hughes'
)
INTERSECT
SELECT book_id FROM translated
WHERE translator_id = (
    SELECT id from translators
    WHERE name = 'Margaret Jull Costa'
);
```

Each of the nested queries here finds the IDs of the books for one translator. The INTERSECT keyword is used to intersect the resulting sets and give us the books they have collaborated on.

Questions

Could we use INTERSECT, UNION etc. to perform operations on 3-4 sets?

- Yes, absolutely. To intersect 3 sets, we would have to use the INTERSECT operator twice. An important note — we have to make sure to have the same number and same types of columns in the sets to be combined using INTERSECT, UNION etc.

Groups

- Consider the ratings table. For each book, we want to find the average rating of the book. To do this, we would first need to group ratings together by book and then average the ratings out for each book (each group).

```
SELECT book_id, AVG(rating) AS average rating
FROM ratings
GROUP BY book_id;
```

In this query, the GROUP BY keyword was used to create groups for each book and then collapse the ratings of the group into an average rating!

- Now, we only want to see the books that are well-rated, with an average rating of over 4.

```
SELECT book_id, ROUND(AVG(rating), 2) AS average rating
FROM ratings
GROUP BY book_id
HAVING average rating > 4.0;
```

Note that the HAVING keyword is used here to specify a condition for the groups, instead of WHERE (which can only be used to specify conditions for individual rows).

Questions

Is it possible to see the number of ratings given to each book?

- Yes, this would require a slight modification with the use of the COUNT keyword.

```
SELECT book_id, COUNT(rating)
FROM ratings
GROUP BY book_id;
```

Is it also possible to sort the data obtained here?

- Yes, it is. Say we wanted to find the average ratings per well-rated book, ordered in descending order.

```
SELECT book_id, ROUND(AVG(rating), 2) AS average rating
FROM ratings
GROUP BY book_id
HAVING average rating > 4.0
ORDER BY average rating DESC;
```

Fin

- This brings us to the conclusion of Lecture 1 about relating!

LECTURE 2

- Introduction
- Creating a Database Schema
  - Normalizing
  - Relating
  - Questions
- CREATE TABLE
  - Questions
- Data Types and Storage Classes
- Type Affinities
- Adding Types to our Tables
  - Questions
- Table Constraints
  - Questions
- Column Constraints
- Altering Tables
  - Questions
- Fin

Introduction

- In this lecture, we will learn how to design our own database schemas.
- Thus far, we have primarily worked with a database of books that were longlisted for the International Booker Prize. Now, we will look underneath the hood and see what commands can be used to create such a database.

- First, let us open up the database longlist.db from Week 0 on our terminal. As a reminder, this database contained just one table, called longlist. To see a snapshot of the table, we can run

```
SELECT author, title
FROM longlist
LIMIT 5;
```

This gives us the authors and titles from the first 5 rows of the table longlist.

- Here is a MYSQL command (not an SQL keyword) that can shed more light on how this database was created.

```
.schema
```

On running this, we see the SQL statement used to create the table longlist. This shows us the columns inside longlist and the types of data that each column is able to store.

- Next, let's open up the same database from Week 1 on our terminal. This version of longlist.db contained different tables related to each other.
- On running .schema again, we see many commands — one for each table in the database. There is a way to see the schema for a specified table:

```
.schema books
```

Now we see the statement used to create the books table. We are also able to see the columns and data types for each column. For example, the title column takes text and the publisher_id column is an integer.

Creating a Database Schema

- Now that we have seen the schema for an existing database, let us create our own! We are tasked with representing the subway system of the city of Boston through a database schema. This includes the subway stations, the different train lines, and the people who take the trains.

- To break down the question further, we need to decide…
  - what kinds of tables we will have in our Boston Subway database,
  - what columns each of the tables will have, and

- what types of data we should put in each of those columns.

Normalizing

- Observe this initial attempt at creating a table to represent Boston Subway data. This table contains subway rider names, current stations the riders are at and the action performed at the station (like entering and exiting). It also records the fares paid and balance amounts on their subway cards. This table also contains an ID for each rider transaction, which serves as the primary key.

id	name	station	action	fare	balance
1	Charlie	Kendall/MIT	enter	10	5
2	Charlie	Jamaica Plain	exit	5	0
3	Alice	Harvard	enter	10	20
4	Alice	Park Street	exit	5	15
5	Bob	Alewife	enter	10	30
6	Bob	Park Street	exit	10	20

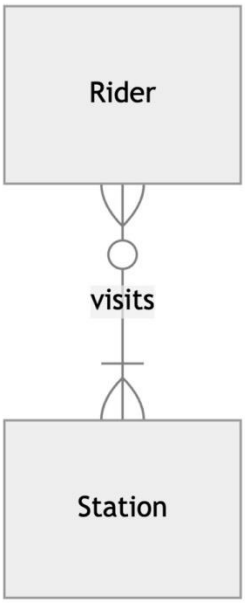
- What redundancies exist in this table?

- We may choose to separate out rider names into a table of its own, to avoid having to duplicate the names so many times. We would need to give each rider an ID that can be used to relate the new table to this one.
- We may similarly choose to move subway stations to a different table and give each subway station an ID to be used as a foreign key here.

- The process of separating our data in this manner is called **normalizing**. When normalizing, we put each entity in its own table—as we did with riders and subway stations. Any information about a specific entity, for example a rider's address, goes into the entity's own table.

Relating

- We now need to decide how our entities (riders and stations) are related. A rider will likely visit multiple stations, and a subway station is likely to have more than one rider. Given this, it will be a many-to-many relationship.
- We can also use an ER diagram to represent this relationship.



Here, we see that every rider must visit at least one station to be considered a rider. A station, though, could have no riders visiting it, because perhaps it is out of order temporarily. However, it is likely that a station has multiple riders visiting it, indicated by the crow's foot in the ER diagram.

Questions

Does the relationship between riders and stations have to be exactly the way described here? For example, why is it okay for a station to have 0 riders?

- It is up to the person designing the database to make decisions about relationships between entities. It is possible to add a constraint that says a station must have at least one rider to be considered a station.

CREATE TABLE

- Now that we have the schema for two of the tables, let's go ahead and create the tables.
- Let us open up a new database called mbta.db — MBTA stands for Massachusetts Bay Transportation Authority and runs the Boston Subway.
- If we run .schema, we will see nothing because no table has been created in this database yet.
- In this database, we run the following command to create the first table for riders:

```
CREATE TABLE riders (
    id,
    name
);
```

On running this, no results appear on the terminal. But if we run .schema again, we will now see the schema for the table riders, as defined by us!

- Similarly, let us create a table for stations as well.

```
CREATE TABLE stations (
    id,
    name,
    line
);
```

Here, we add a column line to store the train line that the station is a part of.

- .schema now shows us the schema for both riders and stations.
- Next, we will create a table to relate these two entities. These tables are often called junction tables, associative entities or join tables!

```
CREATE TABLE visits (
    rider_id,
    station_id
);
```

Each row of this table tells us the station visited by a particular rider.

Questions

Is it necessary to indent the lines within the CREATE TABLE parantheses?

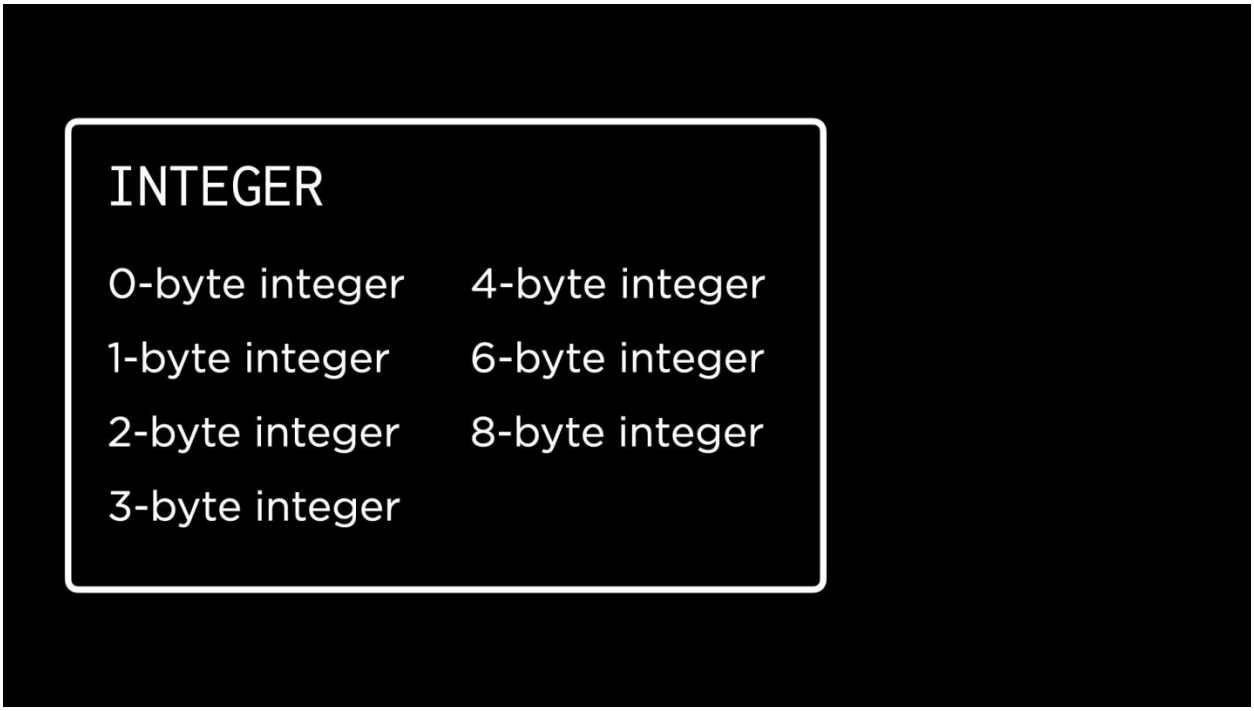
- No, not strictly. However, we indent the column names to adhere to style conventions as always!

Data Types and Storage Classes

- MYSQL has five storage classes:
  - **Null**: nothing, or empty value
  - **Integer**: numbers without decimal points
  - **Real**: decimal or floating point numbers
  - **Text**: characters or strings

- **Blob**: Binary Large Object, for storing objects in binary (useful for images, audio etc.)

- A storage class can hold several data types.
- For example, these are the data types that fall under the umbrella of the Integer storage class.



MYSQL takes care of storing the input value under the right data type. In other words, we as programmers only need to choose a storage class and MYSQL will do the rest!

- Consider this question: what storage class would we use to store fares? Each choice comes with affordances and limitations.

- Integers: We can store a 10 cent fare as the number 10, but that doesn't make it very clear whether the fare is 10 cents or 10 dollars.
- Text: We can store the fare in text, like $0.10. However, now it will be hard to perform mathematical operations like adding up a rider's fares.
- Real: We can store the fare using a floating point number, like 0.10, but it is not possible to store floating point numbers in binary precisely and—depending on how precise we need to be—doing so may lead to miscalculations down the line.

Type Affinities

- It is possible to specify the data type of a column while creating a table.
- However, columns in MYSQL don't always store one particular data type. They are said to have **type affinities**, meaning that they try to convert an input value into the type they have an affinity for.
- The five type affinities in MYSQL are: Text, Numeric (either integer or real values based on what the input value best converts to), Integer, Real and Blob.
- Consider a column with a type affinity for Integers. If we try to insert 25 (the number 25 but stored as text)

into this column, it will be converted into an integer data type.

- Similarly, inserting an integer 25 into a column with a type affinity for text will convert the number to its text equivalent, 25.

Adding Types to our Tables

- To create the tables in our database again, we will first need to drop (or delete) the existing tables.
- Let's try the following commands
- **DROP TABLE** riders;
- **DROP TABLE** stations;
- **DROP TABLE** visits;

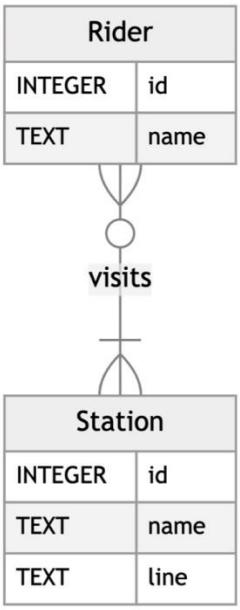
Running these statements gives no output, but .schema shows us that the tables have now been dropped.

- Next, let us create a schema file that can be run to create the tables from scratch. This is an improvement over what we previously did—typing out the CREATE TABLE command for each table—because it allows us to edit and view the entire schema easily.
- Create a file schema.sql. Notice the extension .sql that enables syntax highlighting for SQL keywords in our editor.

- Inside the file, let's type out the schemas again, but with the affinity types this time.

```
CREATE TABLE riders (
    id INTEGER,
    name TEXT
);

CREATE TABLE stations (
    id INTEGER,
    name TEXT,
    line TEXT
);

CREATE TABLE visits (
    rider_id INTEGER,
    station_id INTEGER
);
```

- Now, we read this file within the database to actually create the tables. Here is an updated ER Diagram with the data types included.



Questions

Previously, we were able to query the tables in our database and see the results in a table-like structure. How do we get the same kind of results to show up here?

- We haven't yet added any data to the tables. In Lecture 3, we will see how to insert, update and delete rows in tables that we have created!

Do we have a type affinity for Boolean?

- We don't in MYSQL, but other DBMS's might have this option. A workaround could be to use 0 or 1 integer values to represent booleans.

Table Constraints

- We can use table constraints to impose restrictions on certain values in our tables.
- For example, a primary key column must have unique values. The table constraint we use for this is PRIMARY KEY.
- Similarly, a constraint on a foreign key value is that it must be found in the primary key column of the related table! This table constraint is called, predictably, FOREIGN KEY.
- Let's add primary and foreign key constraints to our schema.sql file.

```
CREATE TABLE riders (
    id INTEGER,
    name TEXT,
    PRIMARY KEY(id)
);

CREATE TABLE stations (
    id INTEGER,
    name TEXT,
    line TEXT,
    PRIMARY KEY(id)
);

CREATE TABLE visits (
```

```
    rider_id INTEGER,
    station_id INTEGER,
    FOREIGN KEY(rider_id) REFERENCES riders(id),
    FOREIGN KEY(station_id) REFERENCES stations(id)
);
```

Notice that we created two primary key columns, the ID for both riders and stations and then referenced these primary keys as foreign keys in the visits table.

- In the visits table, there is no primary key. However, MYSQL gives every table a primary key by default, known as the row ID. Even though the row ID is implicit, it can be queried!
- It is also possible to create a primary key composed of two columns. For example, if we wanted to give visits a primary key composed of both the rider and stations IDs, we could use this syntax

```
CREATE TABLE visits (
    rider_id INTEGER,
    station_id INTEGER,
    PRIMARY KEY(rider_id, station_id)
);
```

In this case, we probably want to allow a rider to visit a station more than once, so we would not move ahead with this approach.

Questions

Is it possible to include our own primary key for the visits table?

- Yes! If, for some reason, an explicit primary key was required for the visits table, we could create an ID column and make it the primary key.

Column Constraints

- A column constraint is a type of constraint that applies to a specified column in the table.
- MYSQL has four column constraints:
  - CHECK: allows checking for a condition, like all values in the column must be greater than 0
  - DEFAULT: uses a default value if none is supplied for a row
  - NOT NULL: dictates that a null or empty value cannot be inserted into the column
  - UNIQUE: dictates that every value in this column must be unique
- An updated schema with these contraints would look like the following:

```
CREATE TABLE riders (
    id INTEGER,
```

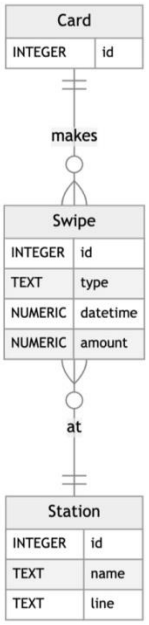
```
    name TEXT,
    PRIMARY KEY(id)
);

CREATE TABLE stations (
    id INTEGER,
    name TEXT NOT NULL UNIQUE,
    line TEXT NOT NULL,
    PRIMARY KEY(id)
);

CREATE TABLE visits (
    rider_id INTEGER,
    station_id INTEGER,
    FOREIGN KEY(rider_id) REFERENCES riders(id),
    FOREIGN KEY(station_id) REFERENCES stations(id)
);
```

The NOT NULL constraint ensures that a station name and line are specified. On the other hand, riders are not mandated to share their names, because there is no constraint applied to rider names. Similarly, each station must have a unique name, as dictated by the UNIQUE constraint.

- Primary key columns and by extension, foreign key columns must always have unique values, so there is no need to explicitly specify the NOT NULL or UNIQUE column constraints. The table constraint PRIMARY KEY includes these column constraints.

Altering Tables

- Consider the following updated ER diagram, where the entity Rider has been swapped out with a new entity Card used to represent CharlieCards. CharlieCards, in the Boston Subway, can be loaded with money and are used to swipe into and sometimes out of stations.



- Notice that a card can be swiped many times in total, but only at one station at a given time.
- The Card entity has an ID, which is also its primary key.
- There is also now an entity Swipe, with an ID of its own and a type. Swipe also records the time at which a card was swiped and the amount that was subtracted (equivalent to the amount of money needed to ride the subway)!
- Now, to implement these changes in our database, we need to first drop the riders table.
- **DROP TABLE** riders;
- Running .schema shows us the updated schema without the riders table.
- Next, we need a swipes table to represent the Swipe entity from our updated ER diagram. We could alter the visits table in the following way.
- **ALTER TABLE** visits
- **RENAME TO** swipes;
- On running .schema we can see that the table visits was renamed to swipes. However, this is not the only change needed. We also need to add some columns, like the type of swipe.
- **ALTER TABLE** swipes
- **ADD COLUMN** swipetype TEXT;

Notice the type affinity TEXT is also mentioned while adding this column.

- We also have the ability to rename a column in an ALTER TABLE command. If we wanted to rename the column swipetype to make it less wordy, perhaps, we could try the following.

```
ALTER TABLE swipes
RENAME COLUMN swipetype TO type;
```

- Finally, we have the ability to drop (or remove) a column.

```
ALTER TABLE swipes
DROP COLUMN type;
```

On running .schema again, we can confirm that the column type was dropped from the table.

- It is also possible to return to the schema file schema.sql that we had originally and simply make these changes there instead of altering tables. The following is an updated schema.sql.

```
CREATE TABLE cards (
    id INTEGER,
    PRIMARY KEY(id)
);

CREATE TABLE stations (
    id INTEGER,
```

```
    name TEXT NOT NULL UNIQUE,
    line TEXT NOT NULL,
    PRIMARY KEY(id)
);

CREATE TABLE swipes (
    id INTEGER,
    card_id INTEGER,
    station_id INTEGER,
    type TEXT NOT NULL CHECK(type IN ('enter', 'exit', 'deposit')),
    datetime NUMERIC NOT NULL DEFAULT CURRENT_TIMESTAMP,
    amount NUMERIC NOT NULL CHECK(amount != 0),
    PRIMARY KEY(id),
    FOREIGN KEY(station_id) REFERENCES stations(id),
    FOREIGN KEY(card_id) REFERENCES cards(id)
);
```

- Let us take a couple of minutes to read through the updated schema and make a note of the things that seem to have changed!
    - The tables cards and swipes are added and the NOT NULL column constraint is used to require some values in swipes.

- The datetime column is given the type affinity numeric — this is because numeric types can store and display date values.
- The foreign key mapping is adjusted as needed, such that card_id is a foreign key referring to the ID of the cards table.
- A default value is assigned to the datetime column so that it automatically picks up the current timestamp if none is supplied. Notice the use of the CURRENT_TIMESTAMP — it returns the year, month, day, hour, minute and second combined into one value.
- There is a check in place to ensure the amount on a swipe is not 0. This is implemented through the column constraint CHECK, which is used with an expression amount != 0 to ensure the value is not 0.
- Similarly, there is a check on type to ensure its value is one of ‘enter’, ‘exit’ and ‘deposit’. This is done because when a CharlieCard is swiped, it is usually for one of these three purposes, so it makes sense to have type assume these values only. Notice the use of the IN keyword to carry out this check! Is there a way to implement this check using the OR operator instead?

Questions

On trying to drop the table riders, an error comes up because we're using the ID of riders as a foreign key. How can the table be dropped in this case?

- Foreign key constraints within the database are checked when dropping a table. Before dropping riders, we would need to first drop the foreign key column rider_id.

How different is the syntax for a different DBMS like MySQL or PostgreSQL?

- Most of the MYSQL syntax definitely applies to other database management systems as well. However, if we tried porting our MYSQL code, some minimal changes would be needed.

If we don't specify a type affinity of a column in MYSQL, what happens?

- The default type affinity is numeric, so the column would get assigned the numeric type affinity.

Fin

- This brings us to the conclusion of Lecture 2 about Designing in SQL! For an interesting story about the origin of the name CharlieCard, read this article from Celebrate Boston.

LECTURE 3

- Introduction
- Database Schema
- Inserting Data
  - Questions
- Other Constraints
- Inserting Multiple Rows
  - Questions
- Deleting Data
  - Questions
- Updating Data
- Fin

Introduction

- Last week, we learned how to create our own database schema. In this lecture, we'll explore how to add, update, and delete data in our databases.
- The Boston MFA (Museum of Fine Arts) is a century-old museum in Boston. The MFA manages a vast collection of historical and contemporary artifacts and artwork. They likely use a database of some kind to store data about their art and artifacts.
- When a new artifact is added to their collection, we can imagine they would insert the corresponding data to their database. Similarly, there are use cases in which data might need to be read, updated or deleted.

- We will focus now on the creation (or insertion) of data in a Boston MFA database.

Database Schema

- Consider this schema that the MFA might use for its collection.

id	title	accession_number	acquired
1	Profusion of flowers	56.257	1956-04-12
2	Farmers working at dawn	11.6152	1911-08-03
3	Spring outing	14.76	1914-01-08

- Each row of data contains the title for a piece of artwork along with the accession_number which is a unique ID used by the museum internally. There is, too, a date indicating when the art was acquired.
- The table contains an ID which serves as the primary key.

- We can imagine that the database administrator of the MFA runs an SQL query to insert each of these pieces of artwork into the table.
- To understand how this works, let us first create a database called mfa.db. Next, we read the schema file schema.sql into the database. This schema file, already given to us, helps us create the table collections.
- To confirm that the table has been created, we can select from the table.
- **SELECT** * **FROM** collections;

  This should give us an empty result, because the table doesn't have any data yet.

Inserting Data

- The SQL statement INSERT INTO is used to insert a row of data into a given table.
- **INSERT INTO** collections (id, title, accession_number, acquired)
- **VALUES** (1, 'Profusion of flowers', '56.257', '1956-04-12');

  We can see that this command requires the list of columns in the table that will receive new data and the values to be added to each column, in the same order.

- Running the INSERT INTO command returns nothing, but we can run a query to confirm that the row is now present in collections.
- **SELECT** * **FROM** collections;
- We can add more rows to the database by inserting multiple times. However, typing out the value of the primary key manually (as 1, 2, 3 etc.) might result in errors. Thankfully, MYSQL can fill out the primary key values automatically. To make use of this functionality, we omit the ID column altogether while inserting a row.
- **INSERT INTO** collections (title, accession_number, acquired)
- **VALUES** ('Farmers working at dawn', '11.6152', '1911-08-03');

We can check that this row has been inserted with an id of 2 by running

SELECT * **FROM** collections;

Notice that the way MYSQL fills out the primary key values is by incrementing the previous primary key—in this case, 1.

Questions

If we delete a row with the primary key 1, will MYSQL automatically assign a primary key of 1 to the next inserted row?

- No, MYSQL actually selects the highest primary key value in the table and increments it to generate the next primary key value.

Other Constraints

- Opening the file schema.sql will pull up the schema for the database.

```
CREATE TABLE collections (
    id INTEGER,
    title TEXT NOT NULL,
    accession_number TEXT NOT NULL UNIQUE,
    acquired NUMERIC,
    PRIMARY KEY(id)
);
```

- It is specified that the accession number is unique. If we try to insert a row with a repeated accession number, we will trigger a error that looks like Runtime error: UNIQUE constraint failed: collections.accession_number (19).
- This error informs us that the row we are trying to insert violates a constraint in the schema—specifically the UNIQUE constraint in this scenario.
- Similarly, we can try to add a row with a NULL title, violating the NOT NULL constraint.

```
INSERT INTO collections (title, accession_number, acquired)
VALUES(NULL, NULL, '1900-01-10');
```

On running this, we will again see an error that looks like Runtime error: NOT NULL constraint failed: collections.title (19).

- In this manner, the schema constraints are guardrails that protect us from adding rows that do not follow the schema of our database.

Inserting Multiple Rows

- We may need to insert more than one row at a time while writing into a database. One way to do this is to separate out the rows using commas in the INSERT INTO command.

```
INSERT INTO table (column0, …)
VALUES
(value0, …),
(value1, …),
…;
```

Inserting multiple rows at once in this manner allows the programmer some convenience. It is also a faster, more efficient way of inserting rows into a database.

- Let us now insert two new paintings into the collections table.

```
INSERT INTO collections (title, accession_number, acquired)
VALUES
('Imaginative landscape', '56.496', NULL),
('Peonies and butterfly', '06.1899', '1906-01-01');
```

The museum may not always know exactly when a painting was acquired, hence it is possible for the acquired value to be NULL, as is the case for the first painting we just inserted.

- To see the updated table, we can select all rows from the table as always.

```
SELECT * FROM collections;
```

- Our data could also be stored in a comma-separated values format, or CSV. Observe in the following example how the values in each row are separated by a comma.

```
id,title,accession_number,acquired
1,Profusion of flowers,56.257,1956-04-12
2,Farmers working at dawn,11.6152,1911-08-03
3,Spring outing,14.76,1914-01-08
4,Imaginative landscape,56.496,
5,Peonies and butterfly,06.1899,1906-01-01
```

- MYSQL makes it possible to import a CSV file directly into our database. To do this, we need to start from scratch. Let us leave this database mfa.db and then remove it.
- We already have a CSV file called mfa.csv that contains the data we need. On opening up this file, we can note that the first row contains the column names, which match exactly with the column names of our table collections as per the schema.
- First, let us create again the database mfa.db and read the schema file as we did earlier.
- Next, we can import the CSV by running a MYSQL command.
- .import *--csv --skip 1 mfa.csv collections*

The first argument, --csv indicates to MYSQL that we are importing a CSV file. This will help MYSQL parse the file correctly. The second argument indicates that the first row of the CSV file (the header row) needs to be skipped, or not inserted into the table.

- We can select all the data from the collections table to see that every painting from mfa.csv has been successfully imported into the table.
- The CSV file we just inserted contained primary key values (1, 2, 3 etc.) for each row of data. However, it is more likely that CSV files we work with will not contain the ID or primary key values. How can we have MYSQL insert them automatically?
- To try this out, let's open up mfa.csv in our codespace and delete the id column from the header row, along with the values in each column. This is what mfa.csv should look like once we finish editing:
- title,accession_number,acquired
- Profusion of flowers,56.257,1956-04-12
- Farmers working at dawn,11.6152,1911-08-03
- Spring outing,14.76,1914-01-08
- Imaginative landscape,56.496,
- Peonies and butterfly,06.1899,1906-01-01
- We will also delete all the rows that are already within the collections table.

- **DELETE FROM** collections;
- Now, we want to import this CSV file into a table. However, the collections table (as per our schema) must have four columns in every row. This new CSV file contains only three columns for every row. Hence, we cannot proceed to import in the same way we did before.
- To successfully import the CSV file without ID values, we will to use a temporary table:
- .import *--csv mfa.csv temp*

  Notice how we don't use the argument --skip 1 with this command. This is because MYSQL is capable of recognizing the very first row of CSV data as the header row, and converts those into the column names of the new temp table.

- We can see the data within the temp table by querying it.
- **SELECT** * **FROM** temp;
- Next, we will select the data (without primary keys) from temp and move it to collections, which was the goal all along! We can use the following command to achieve this.
- **INSERT INTO** collections (title, accession_number, acquired)

- **SELECT** title, accession_number, acquired **FROM** temp;

  In this process, MYSQL will automatically add the primary key values in the id column.

- Just to clean up our database, we can also drop the temp table once we're done moving data.
- **DROP TABLE** temp;

Questions

Can we place columns in specific positions while inserting into a table?

- While we can change the ordering of values in the INSERT INTO command, we usually can't change the ordering of the column names themselves. The order of column names follows the same order used while creating the table.

What happens if one of the multiple rows we are trying to insert violates a table constraint?

- While trying to insert multiple rows into a table, if even one of them violates a constraint, the insertion command will result in an error and none of the rows will be inserted!

After inserting data from the CSV, one of the cells was empty and not NULL. Why did this happen?

- When we imported data from the CSV file, one of the acquired values was missing! This was interpreted as text and hence, read into the table as an empty text value. We can run queries on the table after importing to convert these empty values into NULL if required.

Deleting Data

- We saw previously that running the following command deleted all rows from the table collections. (We don't want to actually run this command now or we'll lose all the data in the table!)
- **DELETE FROM** collections;
- We can also delete rows that match specific conditions. For example, to delete the painting Spring outing from our table collections we can run:
- **DELETE FROM** collections
- **WHERE** title = 'Spring outing';
- To delete any paintings with the date acquired as NULL we can run
- **DELETE FROM** collections
- **WHERE** acquired **IS NULL**;
- As we always do, we will make sure the deletion worked as expected by selecting all data from the table.
- **SELECT** * **FROM** collections;

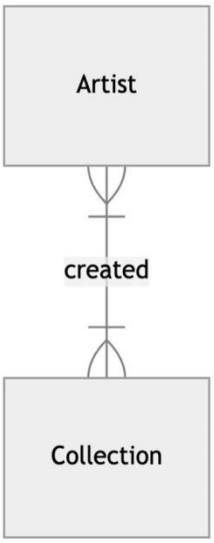
We see that the Spring outing and Imaginative landscape paintings are not in the table anymore.

- To delete rows pertaining to paintings older than 1909, we can run

```
DELETE FROM collections
WHERE acquired < '1909-01-01';
```

Using the < operator here, we are finding the paintings acquired **before** January 1, 1909. These are the paintings that will be deleted on running the query.

- There might be cases where deleting some data could impact the integrity of a database. Foreign key constraints are a good example. A foreign key column references the primary key of a different table. If we were to delete the primary key, the foreign key column would have nothing to reference!
- Consider now an updated schema for the MFA database, containing information not just about artwork but also artists. The two entities Artist and Collection have a many-to-many relationship—a painting can be created by many artists and a single artist can also create many pieces of artwork.



- Here is a database implementing the above ER Diagram.

artists

id	name
1	Li Yin
2	Qian Weicheng
3	Unidentified artist
4	Zhou Chen

created

artist_id	collection_id
1	2
2	3
3	1
4	4

collections

id	title
1	Farmers working ...
2	Imaginative land...
3	Profusion of ...
4	Spring outing

The artists and collections tables have primary keys—the ID columns. The created table references these IDs in its two foreign key columns.

- Given this database, if we choose to delete the unidentified artist (with the ID 3), what would happen to the rows in the table created with an artist_id of 3? Let's try it out.
- After opening up mfa.db, we can now see the updated schema by running the .schema command.
  The created table does indeed have two foreign key constraints, one for the artist ID and one for the collection ID.
- Now, we can try to delete from the artists table.

```
DELETE FROM artists
WHERE name = 'Unidentified artist';
```

On running this, we get an error very similar to ones we have seen before in this class: Runtime error: FOREIGN KEY constraint failed (19). This error notifies us that deleting this data would violate the foreign key constraint set up in the created table.

- How do we ensure that the constraint is not violated? One possibility is to delete the corresponding rows from the created table before deleting from the artists table.

```
DELETE FROM created
```

```
WHERE artist_id = (
    SELECT id
    FROM artists
    WHERE name = 'Unidentified artist'
);
```

This query effectively deletes the artist's *affiliation* with their work. Once the affiliation no longer exists, we can delete the artist's data without violating the foreign key constraint. To do this, we can run

```
DELETE FROM artists
WHERE name = 'Unidentified artist';
```

- In another possibility, we can specify the action to be taken when an ID referenced by a foreign key is deleted. To do this, we use the keyword ON DELETE followed by the action to be taken.
    - ON DELETE RESTRICT: This restricts us from deleting IDs when the foreign key constraint is violated.
    - ON DELETE NO ACTION: This allows the deletion of IDs that are referenced by a foreign key and nothing happens.
    - ON DELETE SET NULL: This allows the deletion of IDs that are referenced by a foreign key and sets the foreign key references to NULL.

- ON DELETE SET DEFAULT: This does the same as the previous, but allows us to set a default value instead of NULL.
- ON DELETE CASCADE: This allows the deletion of IDs that are referenced by a foreign key and also proceeds to cascadingly delete the referencing foreign key rows. For example, if we used this to delete an artist ID, all the artist's affiliations with the artwork would also be deleted from the created table.

- The latest version of the schema file implements the above method. The foreign key constraints now look like

```
• FOREIGN KEY(artist_id) REFERENCES artists(id) ON DELETE CASCADE
• FOREIGN KEY(collection_id) REFERENCES collections(id) ON DELETE CASCADE
```

Now running the following DELETE statement will not result in an error, and will cascade the deletion from the artists table to the created table:

```
DELETE FROM artists
WHERE name = 'Unidentified artist';
```

To check that this cascading deletion worked, we can query the created table:

```
SELECT * FROM created;
```

We observe that none of the rows have an ID of 3 (the ID of the artist deleted from the artists table).

Questions

We just deleted an artist with the ID of 3. Is there any way to make the next inserted row have an ID of 3?

- By default, as we discussed before, MYSQL will select the largest ID present in the table and increment it to obtain the next ID. But we can use the AUTOINCREMENT keyword while creating a column to indicate that any deleted ID should be repurposed for a new row being inserted into the table.

Updating Data

- We can easily imagine scenarios in which data in a database would need to be updated. Perhaps, in the case of the MFA database, we find out that the painting Farmers working at dawn originally mapped to an Unidentified artist was actually created by the artist Li Yin.
- We can use the update command to make changes to say, the affiliation of a painting. Here is the syntax of the update command.

```
UPDATE table
SET column0 = value0, …
WHERE condition;
```

- Let’s change this affiliation for Farmers working at dawn in the created table using the above syntax.

```
UPDATE created
SET artist_id = (
    SELECT id
    FROM artists
    WHERE name = 'Li Yin'
)
WHERE collection_id = (
    SELECT id
    FROM collections
    WHERE title = 'Farmers working at dawn'
);
```

The first part of this query specifies the table to be updated. The next part retrieves the ID of Li Yin to set as the new ID. The last part selects the row(s) in created which will be updated with the ID of Li Yin, which is the painting Farmers working at dawn!

Fin

- This brings us to the conclusion of Lecture 3 about Writing in SQL!

LECTURE 4

- Introduction
- Views
- Simplifying
  - Questions
- Aggregating
  - Questions
- Common Table Expression (CTE)
- Partitioning
  - Questions
- Securing
- Soft Deletions
- Fin

Introduction

- Thus far, we have learned about concepts that allow us to design complex databases and write data into them. Now, we will explore ways in which to obtain views from these databases.
- Let's go back to the database containing books longlisted for the International Booker Prize. Here is a snapshot of tables from this database.

artists

id	name
1	Li Yin
2	Qian Weicheng
3	Unidentified artist
4	Zhou Chen

created

artist_id	collection_id
1	2
2	3
3	1
4	4

collections

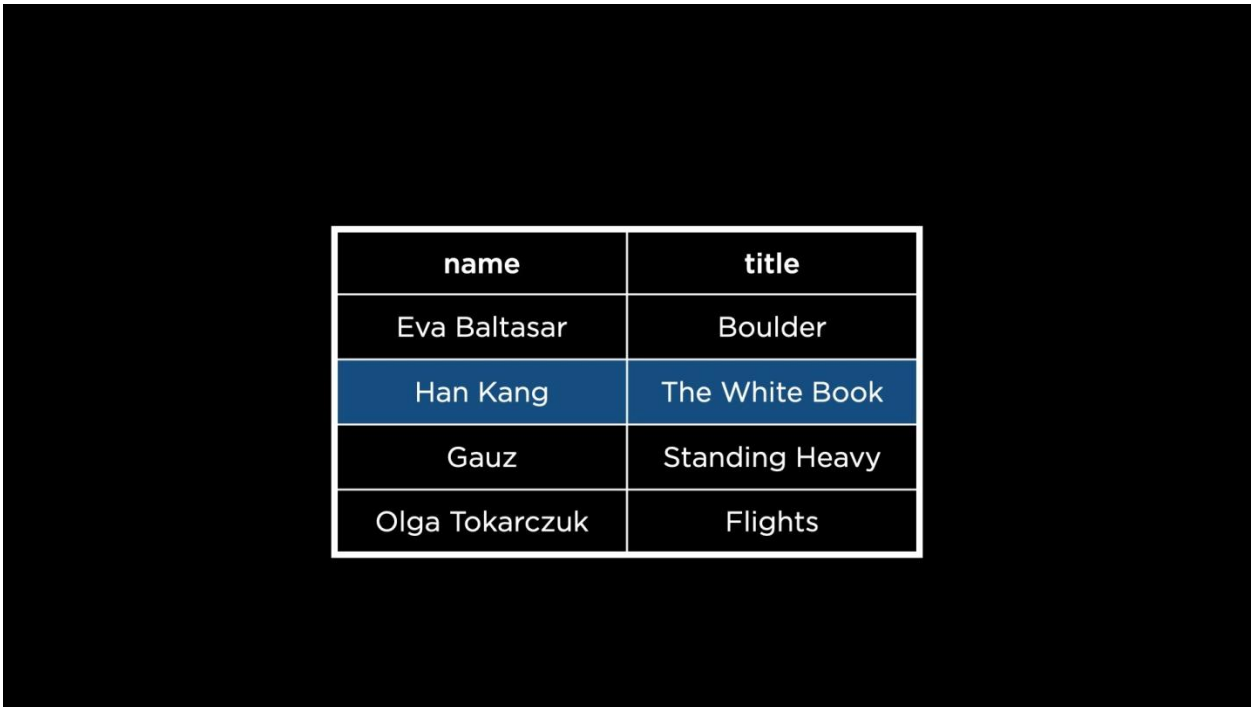
id	title
1	Farmers working ...
2	Imaginative land...
3	Profusion of ...
4	Spring outing

- To find a book written by the author Han Kang, we would need to go each of through the three table above — first finding the author's ID, then the corresponding book IDs and then the book titles. Instead, is there a way to put together related information from the three tables in a single view?
- Yes, we can use the JOIN command in SQL to combine rows from two or more tables based on a related column between them. Here is a visual representation of how these tables could be joined in order to line up authors and their books.

name	id	author_id	book_id	id	title
Eva Baltasar	23	23	1	1	Boulder
Han Kang	31	31	74	74	The White Book
Gauz	27	27	4	4	Standing Heavy
Olga Tokarczuk	58	58	78	78	Flights

This makes it simple to observe that Han Kang authored The White Book.

- One can also imagine removing the ID columns here, such that our view looks like the following.



name	title
Eva Baltasar	Boulder
Han Kang	The White Book
Gauz	Standing Heavy
Olga Tokarczuk	Flights

Views

- A view is a virtual table defined by a query.
- Say we wrote a query to join three tables, as in the previous example, and then select the relevant columns. The new table created by this query can be saved as a view, to be further queried later on.
- Views are useful for:
  - **simplifying**: putting together data from different tables to be queried more simply,
  - **aggregating**: running aggregate functions, like finding the sum, and storing the results,
  - **partitioning**: dividing data into logical pieces,
  - **securing**: hiding columns that should be kept secure. While there are other ways in which

views can be useful, in this lecture we will focus on the above four.

Simplifying

- Let us open up longlist.db on SQL and run the .schema command to verify that the three tables we saw in the previous example are created: authors, authored and books.
- To select the books written by Fernanda Melchor, we would write this nested query.

```
SELECT title FROM books
WHERE id IN (
    SELECT book_id FROM authored
    WHERE author_id = (
        SELECT id FROM authors
        WHERE name = 'Fernanda Melchor'
    )
);
```

- The above query is complex — there are three SELECT queries in the nested query. To simplify this, let us first use JOIN to create a view containing authors and their books.
- In a new terminal, let us connect to longlist.db again, and run the following query.

```
SELECT name, title FROM authors
JOIN authored ON authors.id = authored.author_id
JOIN books ON books.id = authored.book_id;
```

- Observe that it is important to specify how two tables are joined, or the columns they are joined *on*.
- Tip: The primary key column of one table is usually joined to the corresponding foreign key column of the other table!
- Running this will pull up a table containing all the author names next to the titles of the books they have authored.

- To save the virtual table created in the previous step as a view, we need to change the query.

```
CREATE VIEW longlist AS
SELECT name, title FROM authors
JOIN authored ON authors.id = authored.author_id
JOIN books ON books.id = authored.book_id;
```

The view created here is called longlist. This view can now be used exactly as we would use a table in SQL.

- Let us write a query to see all the data within this view.

```
SELECT * FROM longlist;
```

- Using this view, we can considerably **simplify** the query needed to find the books written by Fernanda Melchor.

- **SELECT** title **FROM** longlist **WHERE** name = 'Fernanda Melchor';
- A view, being a virtual table, does not consume much more disk space to create. The data within a view is still stored in the underlying tables, but still accessible through this simplfied view.

Questions

Can we manipulate views to be ordered, or displayed differently?

- Yes, we can order books in a view in much the same way as we can in a table.
  - As an example, let us display the data within the longlist view, ordered by the book titles.
  - **SELECT** name, title
  - **FROM** longlist
  - **ORDER BY** title;
  - We could also have the view itself be ordered. We can do this by including an ORDER BY clause in the query used to create the view.

Aggregating

- In longlist.db we have a table containing individual ratings given to each book. In previous weeks, we saw how to find the average rating of every book, rounded to 2 decimal places.

- **SELECT** book_id, ROUND(**AVG**(rating), 2) **AS** rating
- **FROM** ratings
- **GROUP BY** book_id;
- The results of the above query can be made more useful by displaying the title of every book, and perhaps the year in which each book was longlisted. This information is present in the books table.
- **SELECT** book_id, title, year, ROUND(**AVG**(rating), 2) **AS** rating
- **FROM** ratings
- **JOIN** books **ON** ratings.book_id = books.id
- **GROUP BY** book_id;
    - Here, we use a JOIN to combine information from the ratings and books tables, joining on the book ID column.
    - Notice the order of operations in this query — in particular, the placement of the GROUP BY operation at the end of the query after the two tables are joined.
- This **aggregated** data can be stored in a view.
- **CREATE VIEW** average_book_ratings **AS**
- **SELECT** book_id **AS** id, title, year, ROUND(**AVG**(rating), 2) **AS** rating
- **FROM** ratings
- **JOIN** books **ON** ratings.book_id = books.id
- **GROUP BY** book_id;

- Now, let us see the data in this view.
- **SELECT** * **FROM** average_book_ratings;

- On adding more data to the ratings table, to obtain an up-to-date aggregate, we need to simply requery the view using a SELECT command like the above!
- Each time a view is created, it gets added to the schema. We can verify this by running .schema to observe that longlist and average_book_ratings are now part of this database's schema.
- To create temporary views that are not stored in the database schema, we can use CREATE TEMPORARY VIEW. This command creates a view that exists only for the duration of our connection with the database.
- To find the average rating of books *per year*, we can use the view we already created.

```
SELECT year, ROUND(AVG(rating), 2) AS rating
FROM average_book_ratings
GROUP BY year;
```

Notice that we select the rating column from average_book_ratings, which already contains the average ratings per book. Next, we group these by year and calculate the average ratings again, which gives us the average rating per year!

- We can store the results in a temporary view.

- **CREATE TEMPORARY VIEW** average_ratings_by_year **AS**
- **SELECT** year, ROUND(**AVG**(rating), 2) **AS** rating **FROM** average_book_ratings
- **GROUP BY** year;

Questions

Can temporary views be used to test whether a query works or not?

- Yes, this is a great use case for temporary views! To generalize a little, temporary views are used when we want to organize data in some way without actually storing that organization long-term.

Common Table Expression (CTE)

- A regular view exists forever in our database schema. A temporary view exists for the duration of our connection with the database. A CTE is a view that exists for a single query alone.
- Let us recreate the view containing average book ratings per year using a CTE instead of a temporary view. First, we need to drop the existing temporary view so that we can reuse the name average_book_ratings.
- **DROP VIEW** average_book_ratings;
- Next, we create a CTE containing the average ratings *per book*. We then use the average ratings per

book to calculate the average ratings *per year*, in much the same way as we did before.

```
WITH average_book_ratings AS (
    SELECT book_id, title, year,
ROUND(AVG(rating), 2) AS rating FROM ratings
    JOIN books ON ratings.book_id = books.id
    GROUP BY book_id
)
SELECT year ROUND(AVG(rating), 2) AS rating
FROM average_book_ratings
GROUP BY year;
```

Partitioning

- Views can be used to partition data, or to break it into smaller pieces that will be useful to us or an application. For example, the website for the International Booker Prize has a page of longlisted books for each year the prize was awarded. However, our database stores all the longlisted books in a single table. For the sake of creating the website, or a different purpose, it might be useful to have a different table (or view) of books for each year.
- Let us create a view to store books longlisted in 2022.

```
CREATE VIEW 2022 AS
SELECT id, title FROM books
WHERE year = 2022;
```

  - We can also see the data in this view.

- **SELECT** * **FROM** 2022;

Questions

Can views be updated?

- No, because views do not have any data in the way that tables do. Views actually pull data from the underlying tables each time they are queried. This means that when an underlying table is updated, the next time the view is queried, it will display updated data from the table!

Securing

- Views can be used to enhance database security by limiting access to certain data.
- Consider a rideshare company's database with a table rides that looks like the following.

rides

id	origin	destination	rider
1	Good Egg Galaxy	Honeyhive Galaxy	Peach
2	Castle Courtyard	Cascade Kingdom	Mario
3	Metro Kingdom	Mushroom Kingdom	Luigi
4	Seaside Kingdom	Deep Woods	Bowser

- If we were to give this data to an analyst, whose job is to find the most popular ride routes, it would be irrelevant and indeed, not secure to give them the names of individual riders. Rider names are likely categorized as Personally Identifiable Information (PII) which companies are not allowed to share indiscriminately.
- Views can be handy in this situation — we can share with the analyst a view containing the origin and destination of rides, but not the rider names.
- To try this out, let us open rideshare.db in our terminal. Running .schema should reveal one table called rides in this database.
- We can create a view with the relevant columns, while omitting the rider column altogether. But we

will go one step further here, and create a rider column to display an anonymous rider for each row in the table. This will indicate to the analyst that while we have rider names in the database, the names have been anonymized for security.

- **CREATE VIEW** analysis **AS**
- **SELECT** id, origin, destination, 'Anonymous' **AS** rider
- **FROM** rides;
    - We can query this view to ensure that it is secure.
    - **SELECT** * **FROM** analysis;
- Although we can create a view that anonymizes data, SQL does not allow access control. This means that our analyst could simply query the original rides table and see all the rider names we went to great lengths to omit in the analysis view.

Soft Deletions

- As we saw in previous weeks, a soft deletion involves marking a row as deleted instead of removing it from the table.
- For example, a piece of art called Farmers working at dawn is marked as deleted from the collections table by changing the value in the deleted column from 0 to 1.

collections

id	title	deleted
1	Farmers working ...	1
2	Imaginative land...	0
3	Profusion of ...	0
4	Peonies and ...	0

- We can imagine creating a view to display only the art that is not deleted.
- To try this, let us open mfa.db in our terminal. The collections table does not have a deleted column yet, so we need to add it. The default value here will be 0, to indicate that the row is not deleted.

```
ALTER TABLE collections
ADD COLUMN deleted INTEGER DEFAULT 0;
```

- Now, let us perform a soft delete on the artwork Farmers working at dawn, by updating it to have 1 in the deleted column.

```
UPDATE collections
SET deleted = 1
WHERE title = 'Farmers working at dawn';
```

- We can create a view to display information about the rows that are not deleted.

```
CREATE VIEW current_collections AS
SELECT id, title, accession_number, acquired
FROM collections
WHERE deleted = 0;
```

  - We can display the data in this view to verify that Farmers working at dawn is not present.

```
SELECT * FROM current_collections;
```

  - On soft deletion of a row from the underlying table collections, it will be removed from the current_collections view on any further querying.
- We already know that it is not possible to insert data into or delete data from a view. However, we can set up a trigger that inserts into or deletes from the underlying table! The INSTEAD OF trigger allows us to do this.

```
CREATE TRIGGER delete
INSTEAD OF DELETE ON current_collections
FOR EACH ROW
BEGIN
    UPDATE collections SET deleted = 1
    WHERE id = OLD.id;
END;
```

  - Every time we try to delete rows from the view, this trigger will instead update

the deleted column of the row in the underlying table collections, thus completing the soft deletion.

  - We use the keyword OLD within our update clause to indicate that the ID of the row updated in collections should be the same as the ID of the row we are trying to delete from current_collections.

- Now, we can delete a row from the current_collections view.

```
DELETE FROM current_collections
WHERE title = 'Imaginative landscape';
```

We can verify that this worked by querying the view.

```
SELECT * FROM current_collections;
```

- Similarly, we can create a trigger that inserts data into the underlying table when we try to insert it into a view.
- There are two situations to consider here. We could be trying to insert into a view a row that already exists in the underlying table, but was soft deleted. We can write the following trigger to handle this situation.

```
CREATE TRIGGER insert_when_exists
INSTEAD OF INSERT ON current_collections
FOR EACH ROW
```

```
WHEN NEW.accession_number IN (
    SELECT accession_number FROM collections
)
BEGIN
    UPDATE collections
    SET deleted = 0
    WHERE accession_number =
NEW.accession_number;
END;
```

- The WHEN keyword is used to check if the accession number of the artwork already exists in the collections table. This works because an accession number, as we know from previous weeks, uniquely identifies every piece of art in this table.
- If the artwork does exist in the underlying table, we set its deleted value to 0, indicating a reversal of the soft deletion.

- The second situation occurs when we are trying to insert a row that does not exist in the underlying table. The following trigger handles this situation.

```
CREATE TRIGGER insert_when_new
INSTEAD OF INSERT ON current_collections
FOR EACH ROW
WHEN NEW.accession_number NOT IN (
    SELECT accession_number FROM collections
)
```

```
BEGIN
    INSERT INTO collections (title, accession_number, acquired)
    VALUES (NEW.title, NEW.accession_number, NEW.acquired);
END;
```

- When the accession number of the inserted data is not already present within collections, it inserts the row into the table.

Fin

- This brings us to the conclusion of Lecture 4 about Viewing in SQL!

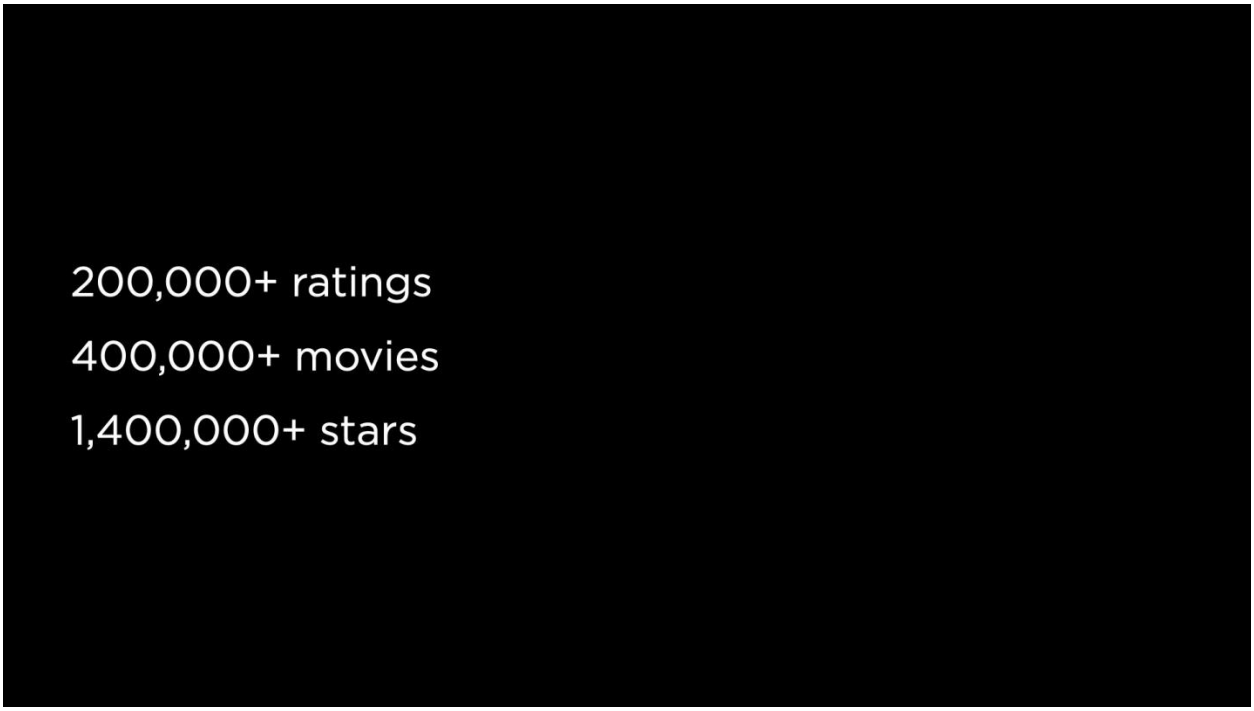
LECTURE 5

- Introduction
- Index
  - Questions

- Index across Multiple Tables
- Space Trade-off
- Time Trade-off
- Partial Index
  - Questions
- Vacuum
  - Questions
- Concurrency
  - Transactions
  - Race Conditions
  - Questions
- Fin

Introduction

- This week, we will learn how to optimize our SQL queries, both for time and space. We will also learn how to run queries concurrently.
- We will do all of this in the context of a new database — the Internet Movies Database, or IMDb as it is more popularly known. Our SQL database is compiled from the large online database of movies you may have seen before at imdb.com.
- Take a look at these statistics to get a sense of how big this database is! It has much more data than any of the other databases we have worked with so far.



- Here is the ER Diagram detailing the entities and their relationships.

- Let us open up this database called movies.db in SQL.
- .schema shows us the tables created in this database. To implement the many-to-many relationship between the entities Person and Movie from the ER Diagram, we have a joint table here called stars that references the ID columns of
  both people and movies as foreign key columns!
- To peek into the movies table, we can select from the table and limit results.
- **SELECT** * **FROM** movies **LIMIT** 5;
- To find the information pertaining to the movie Cars, we would run the following query.
- **SELECT** * **FROM** movies
- **WHERE** title = 'Cars';
  - Say we want to find how long it took for this query to run. SQL has a command .timer on that enables us to time our queries.
  - On running the above query to find Cars again, we can see three different time measurements displayed along with the results.
  - real time indicates the stopwatch time, or the time between executing the query and obtaining the results. This is the measure of time we will focus on. The time taken to execute this query during lecture was roughly a tenth of a second!

- Under the hood, when the query to find Cars was run, we triggered a **scan** of the table movies — that is, the table movies was scanned top to bottom, one row at a time, to find all the rows with the title Cars.
- We can optimize this query to be more efficient than a scan. In the same way that textbooks often have an index, databases tables can have an index as well. An index, in database terminology, is a structure used to speed up the retrieval of rows from a table.
- We can use the following command to create an index for the title column in the movies table.
- **CREATE INDEX** title_index **ON** movies (title);
    - After creating this index, we run the query to find the movie titled Cars again. On this run, the time taken is significantly shorter (during lecture,almost eight times faster than the first run)!
- In the previous example, once the index was created, we just assumed that SQL would use it to find a movie. However, we can also explicitly see this by using a SQL command EXPLAIN QUERY PLAN before any query.
- To remove the index we just created, run:
- **DROP INDEX** title_index;
    - After dropping the index, running EXPLAIN QUERY PLAN again with the SELECT query

will demonstrate that the plan would revert to scanning the entire database.

Questions

Do databases not have implicit algorithms to optimize searching?

- They do, for some columns. In SQL and most other database management systems, if we specify that a column is a primary key, an index will automatically be created via which we can search for the primary key. However, for regular columns like title, there would be no automatic optimization.

Would it be advisable to create a different index for every column in case we need it?

- While that seems useful, there are trade-offs with space and the time it takes to later insert data into tables with an index. We will see more on this shortly!

Index across Multiple Tables

- We would run the following query to find all the movies Tom Hanks starred in.

```
SELECT title FROM movies
WHERE id IN (
    SELECT movie_id FROM stars
    WHERE person_id = (
```

```
        SELECT id FROM people
        WHERE name = 'Tom Hanks'
    )
);
```

- To understand what kind of index could help speed this query up, we can run EXPLAIN QUERY PLAN ahead of this query again. This shows us that the query requires two scans — of people and stars. The table movies is not scanned because we are searching movies by its ID, for which an index is automatically created by SQL!
- Let us create the two indexes to speed this query up.

```
CREATE INDEX person_index ON stars (person_id);
CREATE INDEX name_index ON people (name);
```

- Now, we run EXPLAIN QUERY PLAN with the same nested query. We can observe that
    - all the scans are now searches using indexes, which is great!
    - the search on the table people uses something called a COVERING INDEX
- A covering index means that all the information needed for the query can be found within the index itself. Instead of two steps:
    1. looking up relevant information in the index,

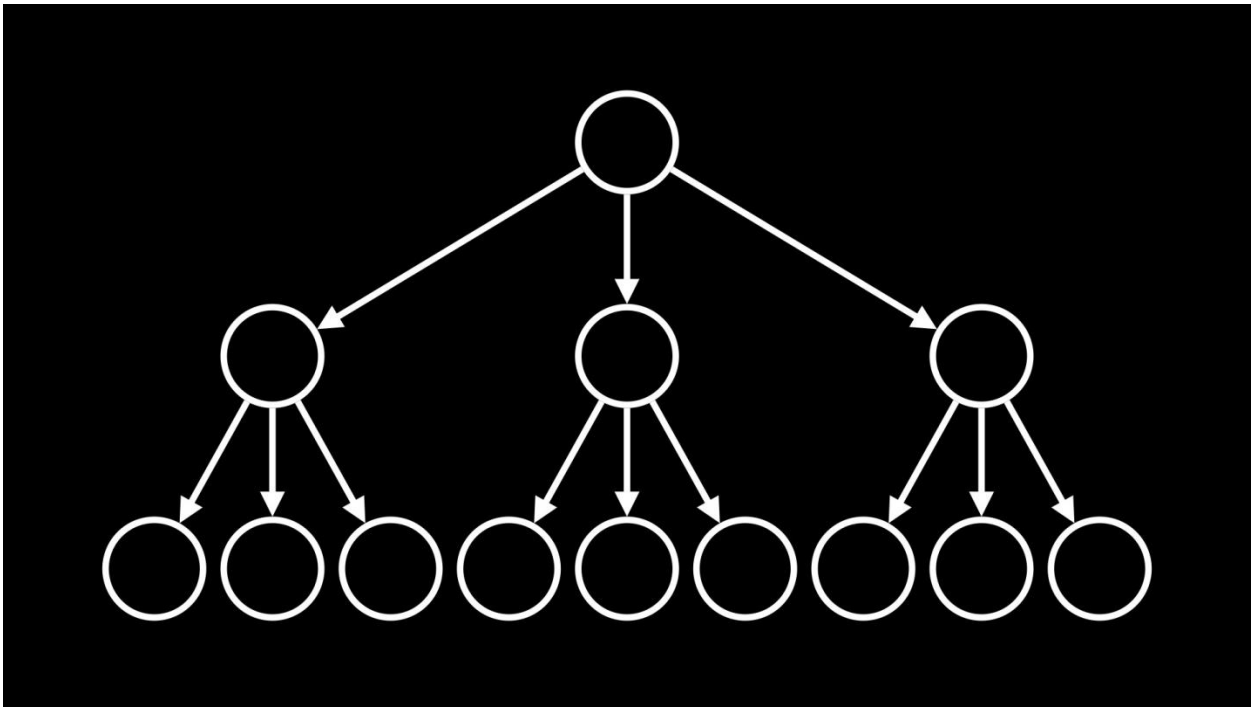
2. using the index to then search the table, a covering index means that we do our search in one step (just the first one).

- To have our search on the table stars also use a covering index, we can add movie_id to the index we created for stars. This will ensure that the information being looked up (movie ID) *and* the value being searched on (person ID) are both be in the index.
- First, let us drop the existing implementation of our index on the stars table.
- **DROP INDEX** person_index;
- Next, we create the new index.
- **CREATE INDEX** person_index **ON** stars (person_id, movie_id);
- Running the following will demonstrate that we now have two covering indexes, which should result in a much faster search!
- **EXPLAIN** QUERY PLAN
- **SELECT** title **FROM** movies **WHERE** id **IN** (
- **SELECT** movie_id **FROM** stars **WHERE** person_id = (
- **SELECT** id **FROM** people **WHERE** name = 'Tom Hanks'
-)
-);
- Making sure that we have run .timer on let us execute the above query to find all the movies Tom Hanks has

starred in, and observe the time it takes to run. The query now runs a *lot* faster than it did without indexes (in lecture, an order of magnitude faster)!

Space Trade-off

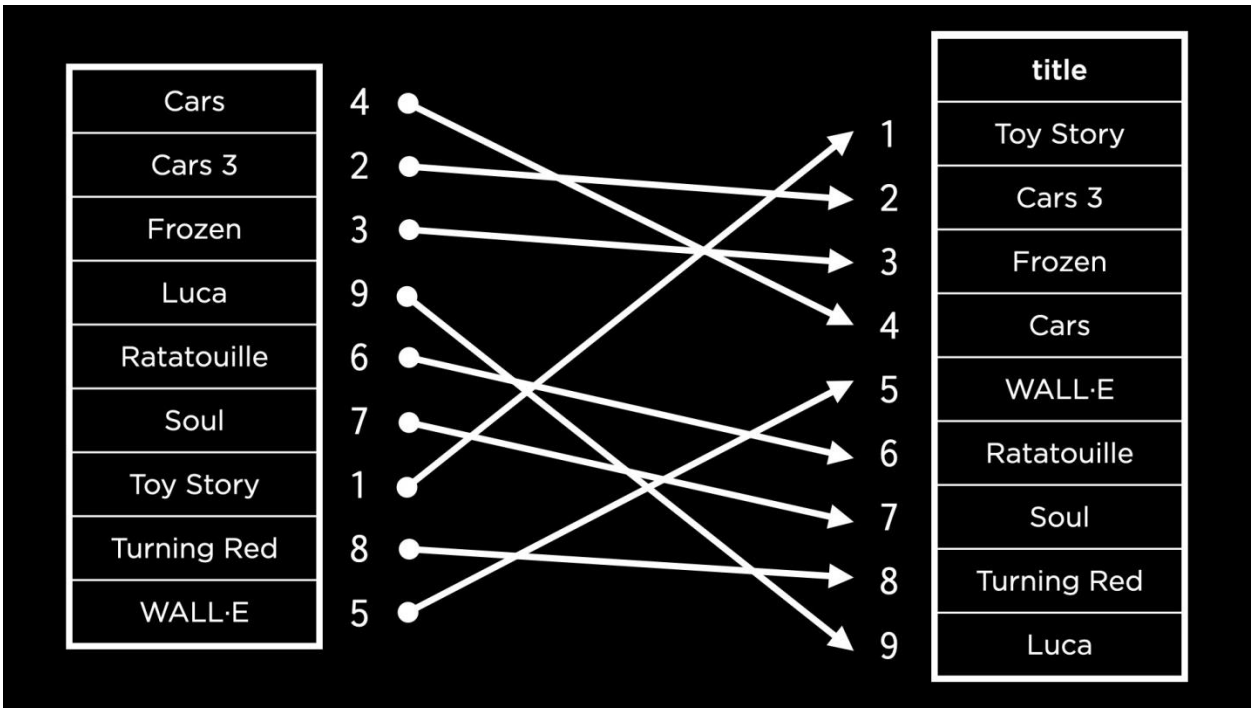
- Indexes seem incredibly helpful, but there are trade-offs associated — they occupy additional space in the database, so while we gain query speed, we do lose space.
- An index is stored in a database as a data structure called a B-Tree, or balanced tree. A tree data structure looks something like:



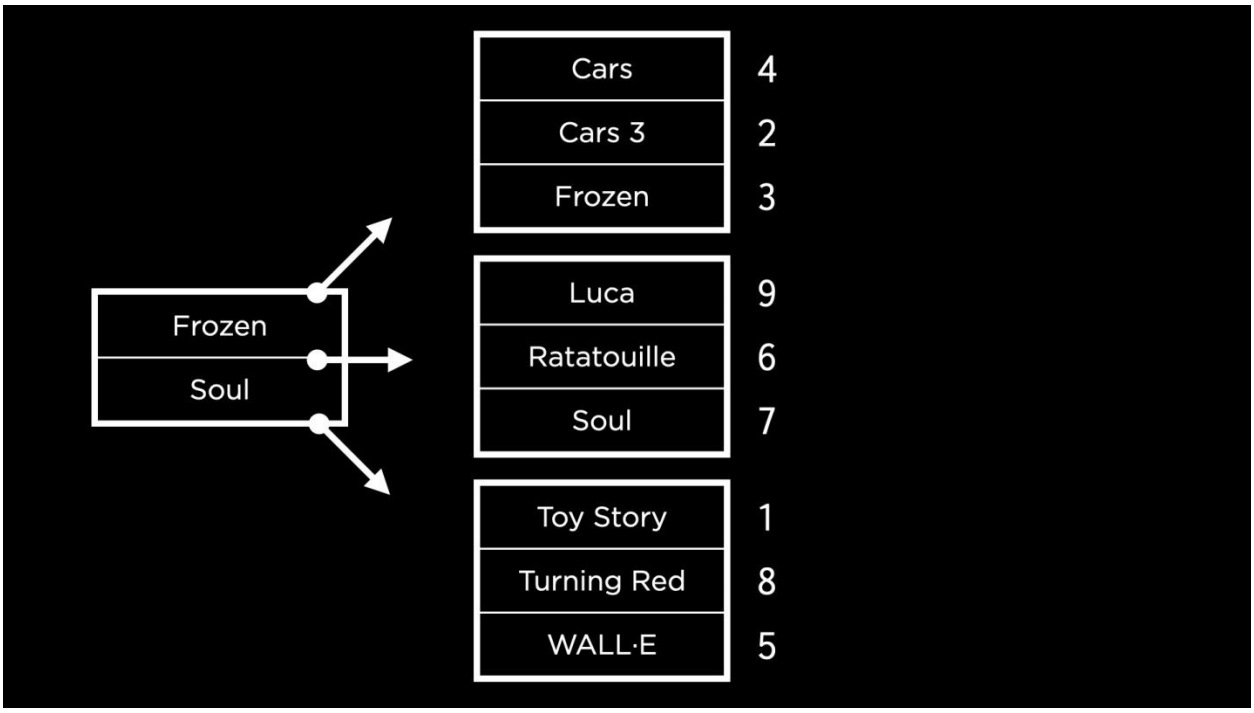
  - Notice that the tree has many **nodes**, each connected to a few others by arrows. The root

node, or the node from which the tree originates, has three **children**. Some nodes at the edge of the tree do not point to any other nodes. These are called **leaf** nodes.

- Let us consider how an index is created for the title column of the table movies. If the movie titles were sorted alphabetically, it would be a lot easier to find a particular movie by using binary search.
- In this case, a copy is made of the titles column. This copy is sorted and then linked back to the original rows within the movies table by pointing to the movie IDs. This is visualized below.



- While this helps us visualize the index for this column easily, in reality, the index is not a single column but is broken up into many nodes. This is because if the database has a lot of data, like our IMDb example, storing one column all together in memory might not be feasible.
- If we have multiple nodes containing sections of the index, however, we also need nodes to navigate to the right sections. For example, consider the following nodes. The left-hand node directs us to the right section of the index based on whether the movie title comes before Frozen, between Frozen and Soul, or after Soul alphabetically!



- The above representation is a B-tree! This is how indexes are stored in SQL.

Time Trade-off

- Similar to the space trade-off we discussed earlier, it also takes longer to insert data into a column and then add it to an index. Each time a value is added to the index, the B-tree needs to be traversed to figure out where the value should be added!

Partial Index

- This is an index that includes only a subset of rows from a table, allowing us to save some space that a full index would occupy.
- This is especially useful when we know that users query only a subset of rows from the table. In the case of IMDb, it may be that the users are more likely to query a movie that was just released as opposed to a movie that is 15 years old. Let's try to create a partial index that stores the titles of movies released in 2023.

```
CREATE INDEX recents ON movies (titles)
WHERE year = 2023;
```

- We can check that searching for movies released in 2023 uses the new index.

```
EXPLAIN QUERY PLAN
SELECT title FROM movies
```

- **WHERE** year = 2023;

This shows us that the movies table is scanned using the partial index.

Questions

Are indexes saved in the schema?

- Yes, in SQL, they are! We can confirm this by running .schema and we will see the indexes created listed in the database schema.

Vacuum

- There are ways to delete unused space in our database. SQL allows us to vacuum data — this cleans up previously deleted data (that is actually not deleted, but just marked as space being available for the next INSERT).
- To find the size of movies.db on the terminal, we can use a Unix command
- du -b movies.db
- In lecture, this command showed us that the size of the database is something like 158 million bytes, or 158 megabytes.
- We can now connect to our database and drop an index we previously created.
- **DROP INDEX** person_index;

- Now, if we run the Unix command again, we see that the size of the database has not decreased! To actually clean up the deleted space, we need to vacuum it. We can run the following command in SQL.
- **VACUUM**;

  This might take a second or two to run. On running the Unix command to check the size of the database again, we can should see a smaller size. Once we drop all the indexes and vacuum again, the database will be considerably smaller than 158 MB (in lecture, around 100 MB).

Questions

Is it possible to vacuum faster?

- Each vacuum can take a different amount of time, depending on the amount of space we are trying to vacuum and how easy it is to find the bits and bytes that need to be freed up!

If a query to delete some rows doesn't actually delete them, but only marks them as deleted, could we still retrieve these rows?

- People trained in forensics are able to find data we think is deleted but is actually still on our computers. In the case of SQL, after performing a vacuum, it would not be possible to find deleted rows again.

Concurrency

- Thus far, we have seen how to optimize single queries. Now, we will look at how to allow not just one query, but multiple at a time.
- Concurrency is the simultaneous handling of multiple queries or interactions by the database. Imagine a database for a website, or a financial service, that gets a lot of traffic at the same time. Concurrency is particularly important in these cases.
- Some database transactions can be multi-part. For example, consider a bank's database. The following is a view of the table accounts that stores account balances.

- One transaction could be sending money from one account to the other. For example, Alice is trying to send $10 to Bob.
- To complete this transaction, we would need to add $10 to Bob's account and also subtract $10 from Alice's account. If someone sees the status of the accounts database after the first update to Bob's account but before the second update to Alice's account, they could get an incorrect understanding of the total amount of money held by the bank.

Transactions

- To an outside observer, it should seem like the different parts of a transaction happen all at once. In database terminology, a transaction is an individual unit of work — something that cannot be broken down into smaller pieces.
- Transactions have some properties, which can be remembered using the acronym ACID:
  - **atomicity**: can't be broken down into smaller pieces,
  - **consistency**: should not violate a database constraint,
  - **isolation**: if multiple users access a database, their transactions cannot interfere with each other,

- **durability**: in case of any failure within the database, all data changed by transactions will remain.

- Let's open up bank.db in our terminal so we can implement a transaction for transferring money from Alice to Bob!
- First, we want to see the data already in the accounts table.
- **SELECT** * **FROM** accounts;

We note here that Bob's ID is 2 and Alice's ID is 1, which will be useful for our query.

- To move $10 from Alice's account to Bob's, we can write the following transaction.
- **BEGIN** TRANSACTION;
- **UPDATE** accounts **SET** balance = balance + 10 **WHERE** id = 2;
- **UPDATE** accounts **SET** balance = balance - 10 **WHERE** id = 1;
- **COMMIT**;

Notice the UPDATE statements are written in between the commands to begin the transaction and to commit it. If we execute the query after writing the UPDATE statements, but without committing, neither of the two UPDATE statements will be run! This helps keep the transaction **atomic**. By updating

our table in this way, we are unable to see the intermediate steps.

- If we tried to run the above transaction again — Alice tries to pay Bob another $10 — it should fail to run because Alice's account balance is at 0.
  (The balance column in accounts has a check constraint to ensure that it has a non-negative value. We can run .schema to check this.)
- The way we implement reverting the transaction is using ROLLBACK. Once we begin a transaction and write some SQL statements, if any of them fail, we can end it with a ROLLBACK to revert all values to their pre-transaction state. This helps keep transactions **consistent**.

```
- BEGIN TRANSACTION;
- UPDATE accounts SET balance = balance + 10
  WHERE id = 2;
- UPDATE accounts SET balance = balance - 10
  WHERE id = 1; -- Invokes constraint error
- ROLLBACK;
```

Race Conditions

- Transactions can help guard against race conditions.
- A race condition occurs when multiple entities simultaneously access and make decisions based on a shared value, potentially causing inconsistencies in

the database. Unresolved race conditions can be exploited by hackers to manipulate the database.

- In the lecture, an example of a race condition is discussed wherein two users working together can exploit momentary inconsistencies in the database to rob the bank.
- However, transactions are processed in **isolation** to avoid the inconsistencies in the first place. Each transaction dealing with similar data from our database will be processed sequentially. This helps prevent the inconsistencies that an adversarial attack can exploit.
- To make transactions sequential, SQL and other database management systems use **locks** on databases. A table in a database could be in a few different states:
    - **UNLOCKED**: this is the default state when no user is accessing the database,
    - **SHARED**: when a transaction is reading data from the database, it obtains shared lock that allows other transactions to read simultaneously from the database,
    - **EXCLUSIVE**: if a transaction needs to write or update data, it obtains an exclusive lock on the database that does not allow other transactions to occur at the same time (not even a read)

Questions

How do we decide when a transaction can get an exclusive lock? How do we prioritize different kinds of transactions?

- Different algorithms could be used to make these decisions. For example, we could always choose the transaction that came first. If an exclusive transaction is needed, no other transaction can run at the same time, which is a necessary downside to ensure consistency of the table.

What is the granularity of locking? Do we lock a database, a table or a row of a table?

- This depends on the DBMS. In SQL, we can actually do this by running an exclusive transaction as below:
- **BEGIN EXCLUSIVE** TRANSACTION;

  If we do not complete this transaction now, and try to connect to the database through a different terminal to read from the table, we will get an error that the database is locked! This, of course, is a very coarse way of locking because it locks the entire database. Because SQL is coarse in this manner, it has a module for prioritizing transactions and making sure an exclusive lock is obtained only for the shortest necessary duration.

Fin

- This brings us to the conclusion of Lecture 5 about Optimizing in SQL!